CEFET/RJ

Centro Federal de Educação Tecnológica Celso Suckow da Fonseca

Bacharelado em Ciência da Computação

GCC1734 – Inteligência Artificial

Aprendizado de Máquina

Nota de Aula

Prof. Eduardo Bezerra

Escola de Informática e Computação

CEFET/RJ

25 de outubro de 2026

Sumário

Objetivos 2

1 Motivação e Taxonomia 2

2 Aprendizado Supervisionado: Dados, Modelos e Funções de Perda 7

3 Generalização e Avaliação 9

4 Modelos Lineares e Aprendizado por Gradiente 14

5 Redes Neurais e Representações Aprendidas 21

6 Síntese e Preparação para Modelos de Linguagem 28

7 Atividades Intra-Classe do Núcleo 29

Apêndices: Consulta e Aprofundamento 31

A Aprofundamento em Modelos Lineares e Características 32

B Aprofundamento em Generalização, Regularização e Métricas 37

C Árvores de Decisão 43

D Métodos Ensemble 49

E Aprofundamento em Redes Neurais 54

F Aprendizado Não Supervisionado 58

G Panorama Comparativo das Famílias de Modelos 62

H Atividades Opcionais de Aprofundamento 64

Referências 68

Objetivos

Ao final do núcleo desta nota, você deve ser capaz de:

- Distinguir aprendizado supervisionado, não supervisionado, autossupervisionado e por reforço, identificando a origem do sinal de aprendizado.
- Formular uma tarefa supervisionada em termos de dados, modelo parametrizado e função de perda.
- Distinguir treinamento, validação e teste; reconhecer vazamento de dados e limitações da generalização.
- Interpretar regressão linear e logística e executar um passo de gradiente descendente.
- Explicar como logits, softmax e entropia cruzada permitem treinar um classificador multiclasse.
- Interpretar um MLP, justificar a não linearidade e distinguir retropropagação de otimização.
- Diagnosticar underfitting e overfitting e propor intervenções coerentes com as evidências.
- Relacionar características projetadas manualmente, representações aprendidas e a futura predição de tokens em modelos de linguagem.

Roteiro de leitura: três encontros. O **núcleo obrigatório** está organizado em três blocos de leitura, seguidos de uma síntese e de atividades presenciais:

- **Encontro 1 – Fundamentos e generalização** (Seções 1–3): tarefas, alvos, parâmetros, perda, divisão dos dados e diagnóstico de erros.
- **Encontro 2 – Modelos lineares e aprendizado por gradiente** (Seção 4): regressão linear, atualização dos parâmetros, classificação probabilística e entropia cruzada.
- **Encontro 3 – Redes neurais e representações** (Seção 5): camadas, não linearidade, retropropagação e representações aprendidas, retomando generalização e regularização.

As atividades da Seção 7 acompanham esses três encontros. Os **Apêndices A–H** são material de consulta e aprofundamento: não integram o roteiro obrigatório nem seus objetivos de avaliação. Não é necessário lê-los para acompanhar a nota seguinte, sobre Modelos de Linguagem.

1 Motivação e Taxonomia

1.1 O que significa aprender uma função?

Uma função associa uma entrada a uma saída. Considere uma máquina cujo consumo de energia depende do tempo de funcionamento. Podemos representar o tempo por x e usar uma função $h(x)$ para prever o consumo. Para cada valor de entrada, a função produz uma estimativa: por exemplo, $h(2)$ é o consumo previsto para duas horas de funcionamento.

Suponha que escolhamos uma família simples de funções, $h_{w,b}(x) = wx+b$. Os números w e b são seus *parâmetros*: w controla quanto a previsão aumenta a cada hora adicional, enquanto b determina a previsão para $x = 0$. Escolher valores diferentes para esses parâmetros produz funções diferentes e, portanto, previsões diferentes para as mesmas entradas.

Inicialmente, os parâmetros podem produzir previsões pouco adequadas. Ao observar exemplos de tempo e consumo medido, podemos comparar as previsões com as medições e modificar os parâmetros para reduzir as discrepâncias. Esse processo é chamado de *ajuste* ou *treinamento* do modelo. A Figura 1 ilustra o resultado de um ajuste, sem detalhar ainda o algoritmo utilizado.

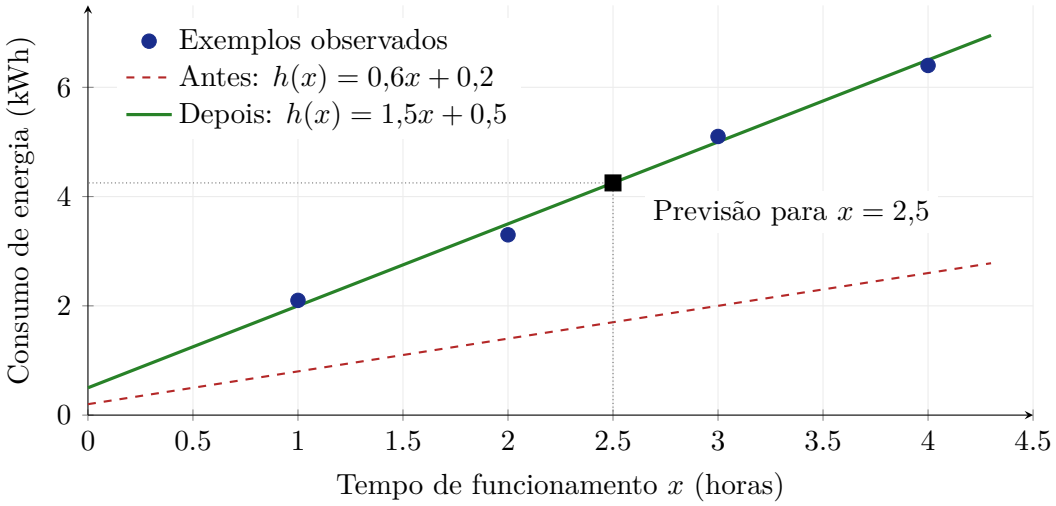


Figura 1: Exemplo ilustrativo de ajuste de uma função. Os pontos representam medições disponíveis para treinamento. A alteração dos parâmetros modifica a inclinação e o deslocamento da reta, aproximando suas previsões dos valores observados. A função ajustada também fornece uma previsão para $x = 2{,}5$, uma entrada que não aparece nos exemplos. Essa previsão não é uma nova medição: sua qualidade precisa ser avaliada com dados adicionais.

Nesse exemplo, *aprender uma função* significa usar os dados para escolher os parâmetros de uma função preditiva dentro da família adotada. Não fornecemos uma resposta independente para cada entrada possível: ajustamos uma regra que pode produzir previsões também para entradas não observadas.

Dizemos que essa função (cujos parâmetros são ajustados a partir de *exemplos*) *aproxima* a relação entre tempo e consumo porque suas previsões procuram representar o comportamento observado, sem exigir coincidência perfeita com todas as medições. Os dados podem conter ruído, e a relação pode não ser exatamente linear. Uma reta bem ajustada aos exemplos disponíveis também não garante boas previsões em outras condições de funcionamento. Essa diferença entre ajustar os dados observados e prever adequadamente casos novos será central no estudo da generalização.

A função linear ilustrada acima foi escolhida apenas para tornar o exemplo simples e visível. Modelos de aprendizado de máquina também podem representar relações não lineares, por meio de curvas, árvores de decisão ou redes neurais, entre outras possibilidades. As entradas podem conter muitas características, e as saídas podem ser valores numéricos, classes ou distribuições de probabilidade. O princípio ilustrado permanece: escolher uma família de modelos e usar dados e um critério de aprendizado para ajustar

uma função dessa família, buscando um comportamento adequado também em situações novas.

1.2 Aprender a partir de dados

Um sistema de aprendizado de máquina (AM) ajusta seu comportamento a partir de dados ou experiência. Em vez de escrever todas as regras para reconhecer spam, por exemplo, fornecemos exemplos e um procedimento para ajustar uma função preditiva. A pergunta central é: *que regularidades dos exemplos observados permitem fazer boas previsões em situações novas?*

O aprendizado por reforço (AR), estudado anteriormente, também é AM. Nele, o agente aprende por interação, com recompensas e transições. Nesta nota, o foco passa para dados, funções preditivas e mecanismos de treinamento que também serão usados em redes neurais e modelos de linguagem.

AM e busca podem ser combinados: uma função de avaliação aprendida pode orientar uma busca; uma política aprendida pode escolher ações. Aprender essa função e procurar um caminho num espaço de estados são problemas distintos, ainda que apareçam no mesmo sistema.

1.3 Do agente às funções que ele aprende

Nos módulos anteriores, usamos o conceito de *agente* para estudar sistemas que percebem um ambiente e escolhem ações. Na busca, examinamos como encontrar sequências de ações para atingir um objetivo. Em ambientes adversariais, consideramos as decisões de outros agentes. No aprendizado por reforço, estudamos como melhorar o comportamento a partir de interação e recompensas.

Nesta nota, mudamos o nível de análise: o foco passa do agente como um todo para o aprendizado de funções a partir de dados. Essas funções podem desempenhar diferentes papéis em um agente: interpretar observações, prever consequências de ações, estimar a qualidade de estados ou escolher ações diretamente.

Por exemplo, um agente que joga pode usar uma função aprendida para avaliar posições e orientar a busca. Um robô pode usar um classificador para reconhecer obstáculos. No aprendizado por reforço com aproximação de funções, a estimativa $Q_{\mathbf{w}}(s, a)$ já era um modelo aprendido usado na escolha de ações. O estudo de AM explicita mecanismos comuns a esses casos: representação das entradas, definição de uma perda, ajuste de parâmetros e generalização.

Um modelo aprendido, entretanto, não precisa constituir sozinho um agente completo. Um classificador de e-mails pode apenas estimar a probabilidade de uma mensagem ser spam. O sistema que o utiliza define como transformar essa previsão em uma ação, como entregar a mensagem, movê-la para outra pasta ou solicitar revisão. A fronteira depende do sistema considerado: um filtro completo também pode ser descrito como um agente simples.

Assim, o conceito de agente continua relevante, mas o vocabulário desta nota enfatiza dados, modelos, previsões e treinamento. Essa distinção será retomada no estudo de Modelos de Linguagem: aprender um modelo capaz de produzir texto e organizar um agente que utiliza esse modelo para decidir e agir são problemas relacionados, com responsabilidades diferentes.

Pergunta 1

Um agente usa uma rede neural para avaliar posições de um jogo e um algoritmo de busca para escolher a próxima jogada. **(a)** Se substituirmos a rede por outra função que também atribui valores às posições, o algoritmo de busca precisa necessariamente mudar? Justifique distinguindo as responsabilidades dos dois componentes. **(b)** Se retirarmos a busca, a função que avalia posições já será suficiente para escolher uma jogada? Descreva o que o sistema ainda precisaria fazer.

1.4 De onde vem o sinal de aprendizado?

Dizer que um sistema aprende a partir de dados ainda deixa uma pergunta em aberto: que informação permite orientar esse aprendizado? Um conjunto de exemplos, por si só, não especifica o que o sistema deve aprender. É necessário definir uma tarefa e um critério para avaliar as soluções produzidas.

Chamaremos de *sinal de aprendizado* a informação que, em conjunto com esse critério, orienta o ajuste do modelo. Ela pode assumir formas diferentes: uma resposta de referência para uma entrada, regularidades presentes nos próprios dados ou recompensas obtidas pela interação com um ambiente. A função de perda e os mecanismos de ajuste serão formalizados nas próximas seções.

Uma maneira de distinguir os paradigmas de aprendizado é examinar como essa informação é obtida e utilizada:

Supervisionado.
Cada entrada de treinamento vem acompanhada de um *alvo*, isto é, uma resposta de referência que o modelo deve aprender a prever. O ajuste é orientado pela comparação entre previsão e alvo. Exemplos incluem prever o preço registrado de um imóvel ou classificar um e-mail previamente rotulado como spam ou legítimo. Os alvos podem vir de anotação humana ou de registros já existentes.

Não supervisionado.
Os exemplos não vêm acompanhados de respostas de referência. O aprendizado é orientado por um critério que procura capturar alguma estrutura dos dados. Em agrupamento, por exemplo, podemos procurar grupos cujos integrantes sejam semelhantes entre si. Portanto, ausência de rótulos não significa ausência de objetivo: é preciso definir que estrutura se deseja encontrar e como avaliá-la. Agrupamento e redução de dimensionalidade são retomados no Apêndice F.

Autossupervisionado.
Os próprios dados fornecem os alvos de uma tarefa de predição. Em um texto, por exemplo, um prefixo pode ser a entrada e o próximo token observado pode ser o alvo. Assim como no aprendizado supervisionado, comparamos uma previsão com uma resposta de referência; a diferença está na construção dos exemplos, que dispensa a anotação externa de cada alvo. Esse é o caso central do pré-treinamento de modelos de linguagem estudado na próxima nota.

Por reforço.
O agente aprende a partir de transições e recompensas obtidas pela interação com o ambiente. A recompensa avalia aspectos dos resultados dessa interação, sem

necessariamente indicar qual ação deveria ter sido escolhida. Como as ações afetam situações futuras, o aprendizado considera o retorno acumulado, e não apenas a recompensa imediata.

Essa classificação diz respeito à organização do aprendizado: quais experiências estão disponíveis, como são construídos os alvos ou critérios e o que orienta o ajuste. Por isso, a arquitetura, isoladamente, não determina o paradigma. Uma mesma rede neural pode ser treinada com exemplos rotulados, com alvos extraídos dos próprios dados ou com recompensas de interação.

As categorias também não são inteiramente independentes. A autossupervisão é frequentemente situada no contexto mais amplo do aprendizado sem anotação externa; aqui, ela é destacada para tornar explícita a origem dos alvos.

Um mesmo texto, diferentes tarefas. Um e-mail pode fornecer: (a) uma entrada acompanhada do rótulo humano "spam"; (b) uma entrada sem rótulo para agrupamento; ou (c) prefixos e próximos tokens extraídos do próprio texto. No terceiro caso, "aprendizado por" pode ser a entrada e "reforço" o alvo. A origem dos alvos muda; os princípios de previsão, perda e ajuste de parâmetros permanecem.

1.5 A hipótese de aprendizagem e seus limites

As subseções anteriores mostraram como dados e um critério podem orientar o ajuste de um modelo. Resta uma questão: por que uma função ajustada aos exemplos disponíveis deveria produzir boas previsões para exemplos novos? No caso da máquina, usamos medições de tempo e consumo para ajustar uma função. Ao aplicar essa função a outra situação de funcionamento, estamos supondo que alguma regularidade observada nas medições anteriores continua válida.

Essa suposição expressa a *hipótese de aprendizagem*: os dados usados no treinamento devem conter regularidades relevantes para as situações em que o modelo será utilizado. No exemplo, esperamos que a relação entre tempo de funcionamento e consumo permaneça suficientemente semelhante. Essa expectativa depende das condições de coleta e de uso dos dados; ela não é garantida pelo algoritmo de treinamento.

Retome a previsão para $x = 2{,}5$ horas na Figura 1. Esse tempo não aparece entre os exemplos observados, mas está dentro do intervalo de uma a quatro horas representado pelos dados. Se a máquina continuar operando em condições semelhantes, pode ser razoável esperar que a função forneça uma estimativa útil. Ainda assim, somente uma nova medição permitirá verificar o erro dessa previsão. Já uma previsão para vinte horas exige mais cautela: fora do intervalo observado, podem surgir efeitos que os exemplos não revelam, como aquecimento ou mudanças no regime de operação.

Mesmo dentro do intervalo observado, as previsões podem deixar de ser adequadas se as condições mudarem. Suponha que as medições de treinamento tenham sido coletadas com a máquina operando sob carga leve, mas que o modelo seja utilizado para prever o consumo sob carga pesada. O mesmo tempo de funcionamento pode então corresponder a um consumo diferente. Se a entrada do modelo contém apenas o tempo, ela não informa essa mudança de condição. Nesse caso, pode ser necessário coletar exemplos representativos dos diferentes regimes e incluir a carga entre as características de entrada.

Repare que *um bom ajuste não garante generalização*. Um modelo pode reproduzir bem as medições usadas no treinamento e ainda apresentar erros elevados em outras situações. Isso pode ocorrer por mudanças nas condições de uso, por poucos exemplos, por uma família de funções inadequada ou por um ajuste excessivo às particularidades dos

dados disponíveis. Chamamos de *generalização* a capacidade de manter um desempenho adequado em exemplos novos. Para avaliá-la, precisamos de dados que não tenham sido usados para ajustar o modelo e que representem as condições de uso pretendidas.

Essa preocupação também se aplica aos demais exemplos da seção: novas estratégias de spam podem comprometer um classificador de e-mails, e mudanças no ambiente podem prejudicar uma política aprendida por reforço. Antes de aplicar AM, portanto, precisamos perguntar não apenas como o modelo será treinado, mas também que situações os dados representam, em quais condições ele será utilizado e como serão avaliadas suas previsões ou decisões.

Pergunta 2

Um modelo foi ajustado com medições de uma máquina operando entre uma e quatro horas, sempre sob carga leve. Agora queremos prever o consumo de outra máquina do mesmo fabricante, após duas horas e meia sob carga leve. **(a)** O fato de o tempo e a carga estarem entre as condições observadas é suficiente para confiar na previsão? Explicite uma suposição adicional necessária. **(b)** Proponha uma coleta de dados que permita investigar se o modelo pode ser usado nessa segunda máquina. **(c)** Se as novas previsões forem ruins, isso demonstra necessariamente que o treinamento na primeira máquina foi mal executado? Justifique.

2 Aprendizado Supervisionado: Dados, Modelos e Funções de Perda

A seção anterior apresentou diferentes formas de organizar o aprendizado. A partir daqui, o corpo principal da nota se concentra no *aprendizado supervisionado*: usamos exemplos de entradas acompanhadas de alvos para ajustar uma função preditiva. Esse recorte permite desenvolver concretamente os conceitos de representação, modelo, perda, otimização e generalização, preparando também o estudo de redes neurais e modelos de linguagem.

Retomando o exemplo da máquina, cada medição fornece um par: o tempo de funcionamento é a entrada, e o consumo observado é o alvo. Durante o treinamento, o modelo recebe esses exemplos e seus parâmetros são ajustados para aproximar as previsões dos valores observados. Ao utilizar o modelo para uma nova previsão, fornecemos a entrada; o alvo correspondente pode ainda não ser conhecido.

A formulação com pares entrada–alvo descreve o problema supervisionado tratado nesta seção, mas não caracteriza toda tarefa de AM. No aprendizado não supervisionado, por exemplo, não dispomos desses alvos e precisamos estabelecer outros critérios para aprender estruturas dos dados. No aprendizado por reforço, o objetivo envolve o retorno obtido pela interação com o ambiente. Já uma tarefa autossupervisionada pode assumir a mesma estrutura de predição apresentada aqui, com a diferença de que seus alvos são construídos a partir dos próprios dados.

2.1 Entradas, alvos e hipóteses

Um conjunto supervisionado pode ser escrito como

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n},$$

onde $\mathbf{x}^{(i)} \in \mathbb{R}^d$ representa a entrada e $y^{(i)}$ é o alvo. Em **regressão**, o alvo é numérico, como um preço. Em **classificação**, pertence a um conjunto de classes, como spam e legítimo.

Escolhemos uma família de funções $\mathcal{H} = \{h_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \Theta\}$, chamada espaço de hipóteses. O vetor $\boldsymbol{\theta}$ reúne os parâmetros aprendidos, como pesos e vieses. O treinamento procura uma hipótese dentro dessa família; ele não procura entre todas as funções possíveis.

Parâmetros, hiperparâmetros e arquitetura. *Parâmetros* são ajustados no treinamento: pesos, vieses e, mais adiante, embeddings. *Hiperparâmetros* orientam o processo ou a escolha do modelo, como taxa de aprendizado, intensidade de regularização e número de camadas. A *arquitetura* especifica a organização das operações e conexões da rede. Essas escolhas são avaliadas com dados de validação; o teste não deve orientá-las.

No exemplo de spam, características podem ser frequências de palavras e quantidade de links. Um modelo linear combina essas características diretamente. Uma rede neural pode aprender transformações intermediárias. Escolher uma representação e uma família de funções determina que padrões poderão ser capturados.

2.2 Como quantificar o erro?

No problema supervisionado considerado aqui, a função de perda $\ell(\hat{y}, y)$ mede a discrepância entre uma previsão $\hat{y}$ e seu alvo y. Para regressão, usaremos o erro quadrático:

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

Essa perda é zero quando a previsão coincide com o alvo e positiva nos demais casos. Como a diferença é elevada ao quadrado, erros de maior magnitude recebem penalidades maiores, independentemente de a previsão estar acima ou abaixo do valor observado.

Retome o exemplo do tempo de funcionamento e do consumo de energia da máquina. Considere a função ajustada $h_{w,b}(x) = wx + b$, com $w = 1{,}5$ e $b = 0{,}5$. Para uma hora de funcionamento, o modelo prevê $\hat{y} = h(1) = 2{,}0$ kWh, enquanto o consumo observado foi $y = 2{,}1$ kWh. A perda nesse exemplo é

$$\ell(2{,}0, 2{,}1) = (2{,}0 - 2{,}1)^2 = (-0{,}1)^2 = 0{,}01.$$

O modelo subestimou o consumo em 0,1 kWh. Se tivesse superestimado pela mesma quantidade, a perda quadrática seria igual.

Para avaliar o ajuste ao conjunto de treinamento, precisamos agregar as perdas dos diferentes exemplos. O custo médio nos exemplos de treinamento é o *risco empírico*:

$$J(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)}\big).$$

Aqui, $\boldsymbol{\theta}$ reúne os parâmetros do modelo; no exemplo da máquina, $\boldsymbol{\theta} = (w, b)$. Quando usamos a perda quadrática, esse custo é chamado de *erro quadrático médio* (MSE, do inglês *mean squared error*).

A tabela a seguir apresenta as previsões e perdas para os quatro exemplos da Figura 1.

Tempo x (h)	Consumo y (kWh)	Previsão $\hat{y}$ (kWh)	Perda $(\hat{y} - y)^2$
1	2,1	2,0	0,01
2	3,3	3,5	0,04
3	5,1	5,0	0,01
4	6,4	6,5	0,01

O risco empírico desse modelo é, portanto,

$$J(w = 1{,}5, b = 0{,}5) = \frac{0{,}01 + 0{,}04 + 0{,}01 + 0{,}01}{4} = 0{,}0175.$$

Como os erros estão em kWh, o erro quadrático médio está em $(\text{kWh})^2$. Note a distinção: a perda avalia uma previsão individual, enquanto o risco empírico resume as perdas no conjunto de treinamento.

Podemos usar esse mesmo critério para comparar diferentes valores dos parâmetros. A função inicial da figura, $h(x) = 0{,}6x + 0{,}2$, produz previsões 0,8, 1,4, 2,0 e 2,6 kWh para as mesmas entradas. Seu custo é

$$J(w = 0{,}6, b = 0{,}2) = \frac{1{,}69 + 3{,}61 + 9{,}61 + 14{,}44}{4} = 7{,}3375.$$

Assim, a função ajustada apresenta um custo de treinamento muito menor. Isso quantifica a melhora ilustrada na figura, mas não demonstra que esses parâmetros sejam os melhores possíveis nem garante bom desempenho em dados novos.

Nesse contexto, treinar significa procurar parâmetros que reduzam esse custo. A função de perda permite comparar candidatos; o procedimento de otimização determina como os parâmetros são modificados para buscar candidatos melhores. A solução obtida depende dos dados, da família de funções e do procedimento de otimização; não há garantia geral de encontrar o mínimo global em redes neurais.

Para classificação probabilística, usaremos a *entropia cruzada*, apresentada na Seção 4.6. Em vez de medir uma diferença entre valores de consumo, ela penaliza o modelo conforme a probabilidade atribuída à classe observada: quanto menor essa probabilidade, maior a perda. Assim, duas previsões que levam à mesma decisão de classe podem receber perdas diferentes. A acurácia, que conta a fração de decisões corretas, não faz essa distinção.

Pergunta 3

Retome os consumos observados de 2,1, 3,3, 5,1 e 6,4 kWh. Outro modelo produz, para essas mesmas entradas, previsões de 2,2, 3,4, 5,2 e 6,5 kWh. **(a)** Antes de calcular, compare os erros com os do modelo anterior, cujo MSE era 0,0175, e estime se o novo MSE será maior ou menor. Justifique sua previsão. **(b)** Calcule as perdas individuais e o risco empírico e verifique sua estimativa. **(c)** Esse resultado é suficiente para afirmar que o novo modelo fará melhores previsões em situações ainda não observadas? Explique que evidência falta.

3 Generalização e Avaliação

Na seção anterior, usamos o erro quadrático médio para comparar funções que preveem o consumo de energia de uma máquina. A função $h(x) = 1{,}5x + 0{,}5$ apresentou menor custo nos exemplos observados do que a função inicial. Essa comparação responde a uma pergunta sobre ajuste: qual função reproduz melhor os dados disponíveis? Entretanto, nosso interesse é usar o modelo para prever o consumo em outras situações. Precisamos, portanto, investigar uma segunda pergunta: o que acontece quando a função recebe entradas de exemplos que não participaram de seu ajuste?

Essa passagem do desempenho nos exemplos usados para aprender ao desempenho em exemplos novos é o problema da *generalização*. Como discutido na Seção 1.5, esperamos que os dados disponíveis contenham regularidades úteis para as condições de uso. A avaliação procura verificar essa expectativa, reconhecendo que nenhum conjunto finito permite garantir o desempenho em todas as situações futuras.

3.1 Treino, validação e teste

Suponha que coletemos novas medições de tempo de funcionamento e consumo da máquina. Se utilizarmos todas elas para ajustar os parâmetros e depois calcularmos o erro nos mesmos exemplos, teremos medido apenas o desempenho de treinamento. Para avaliar a capacidade de previsão em exemplos novos, precisamos separar dados que não serão usados nesse ajuste.

Além disso, o desenvolvimento envolve escolhas: usar uma função linear ou uma família mais flexível, incluir outras características e definir como controlar o treinamento. Se fizermos todas essas escolhas com base nos mesmos dados usados para a avaliação final, também estaremos nos adaptando a esses exemplos. Por isso, distinguimos três papéis para os dados:

- **Treinamento:** usado para calcular a função de custo e ajustar os parâmetros do modelo. No exemplo da máquina, esses dados orientam a escolha dos valores de w e b.
- **Validação:** usado para comparar alternativas durante o desenvolvimento, escolher hiperparâmetros e decidir quando interromper o treinamento. Por exemplo, podemos comparar modelos que usam apenas o tempo com modelos que também recebem a carga de operação.
- **Teste:** reservado para estimar o desempenho do modelo escolhido, depois de concluídas as decisões de desenvolvimento. Seus resultados não devem orientar ajustes de parâmetros, hiperparâmetros ou representação.

O conjunto de validação não participa diretamente das atualizações dos parâmetros, mas influencia a seleção do modelo. Portanto, também podemos nos adaptar excessivamente a ele ao experimentar muitas alternativas. O conjunto de teste fornece uma avaliação separada dessas escolhas, desde que permaneça reservado até o fim do desenvolvimento.

Não existe uma proporção universal para essa divisão. O volume de dados e as condições de uso pretendidas determinam uma separação adequada. Se queremos prever o consumo futuro de uma máquina, pode ser necessário treinar com registros anteriores e avaliar em registros posteriores. Se queremos usar o modelo em máquinas diferentes, separar medições aleatoriamente pode ser insuficiente: uma avaliação mais apropriada pode reservar máquinas inteiras para validação e teste.

Essa separação precisa ser respeitada durante todo o desenvolvimento. Se consultarmos repetidamente o resultado de teste e modificarmos o modelo para melhorá-lo, nossas escolhas passarão a depender também desses dados. O teste deixará, então, de representar uma avaliação separada do processo de seleção. Por isso, as decisões de desenvolvimento devem ser orientadas pelo treinamento e pela validação, mantendo o teste reservado para o final.

3.2 Underfitting, overfitting e diagnóstico

Depois de separar os dados, podemos comparar o desempenho de treinamento com o de validação. Essa comparação ajuda a investigar por que um modelo não produz as previsões desejadas. Dois conceitos importantes são *underfitting*, ou ajuste insuficiente, e *overfitting*, ou ajuste excessivo às particularidades do treinamento.

No exemplo da máquina, suponha que o consumo cresça de forma acentuadamente curva com o tempo. Uma função linear pode não conseguir representar essa relação, mesmo depois de um treinamento adequado. Se isso resultar em erros elevados tanto no treino quanto na validação, teremos evidência compatível com underfitting: a família de funções adotada pode ser limitada demais para os padrões relevantes.

Agora suponha que escolhamos uma função muito flexível, capaz de passar exatamente por todas as medições de treinamento. Ela pode acompanhar também pequenas flutuações causadas por ruído, em vez de capturar apenas a tendência de consumo. Se o erro de treino ficar muito baixo, mas o erro de validação for elevado, teremos um sinal de overfitting. Coincidir com todos os exemplos observados não é, por si só, o objetivo do aprendizado.

Considere os resultados ilustrativos abaixo. Todos foram calculados com o mesmo erro quadrático médio e sobre as mesmas divisões dos dados.

Modelo	**MSE de treino**	**MSE de validação**	**Interpretação inicial**
A	1,20	1,25	Erros próximos; investigar ajuste insuficiente, otimização e qualidade das entradas.
B	0,03	0,05	Desempenho promissor, se esses erros forem adequados à aplicação.
C	0,001	0,90	Grande diferença entre treino e validação; investigar overfitting e diferenças entre os dados.

Esses valores não permitem um diagnóstico isolado do contexto. Para considerar o erro do modelo A elevado, precisamos de uma referência, como o desempenho de uma previsão simples e a tolerância da aplicação. Da mesma forma, a diferença observada no modelo C pode decorrer de overfitting, mas também de condições de operação presentes na validação e ausentes no treino. O diagnóstico exige examinar os dados, o processo de treinamento e os erros produzidos.

As intervenções devem acompanhar essa investigação. Capacidade insuficiente pode justificar uma família de funções mais expressiva; treinamento inadequado pode exigir mudanças no procedimento de otimização; overfitting pode motivar regularização ou mais dados representativos. Se a carga de operação afeta o consumo e não aparece na entrada, aumentar apenas a flexibilidade da função pode não resolver o problema.

3.3 Viés, variância e regularização: a intuição necessária

Os conceitos de viés e variância usados em AM estão diretamente relacionados aos conceitos de viés e variância de um estimador estudados em Estatística. Um estimador é uma regra que usa uma amostra para estimar uma quantidade desconhecida. Como diferentes amostras podem produzir estimativas diferentes, estudamos tanto o desvio de

sua estimativa média em relação à quantidade desejada quanto sua variabilidade entre amostras.

No aprendizado supervisionado, o procedimento de treinamento também transforma uma amostra em uma estimativa: a partir do conjunto de treinamento $\mathcal{D}$, obtemos uma função ajustada, que representaremos por $\hat{h}_{\mathcal{D}}$. Para uma entrada fixa x, a previsão $\hat{h}_{\mathcal{D}}(x)$ pode ser interpretada como um estimador. Em regressão com perda quadrática, a quantidade que procuramos estimar é a resposta média para essa entrada, $f(x) = \mathbb{E}[Y \mid X = x]$. No exemplo da máquina, $f(2{,}5)$ representa o consumo médio em situações com duas horas e meia de funcionamento, nas condições consideradas.

Imagine repetir a coleta de dados várias vezes, mantendo o tamanho da amostra e as condições de coleta, e treinar um modelo em cada conjunto obtido. Para o mesmo tempo de funcionamento, cada modelo poderá produzir uma previsão diferente. A média dessas previsões permite definir o **viés**:

$$\text{Viés}(x) = \mathbb{E}_{\mathcal{D}}\big[\hat{h}_{\mathcal{D}}(x)\big] - f(x).$$

A expectativa é tomada sobre os possíveis conjuntos de treinamento. Portanto, viés não é simplesmente o erro de uma previsão isolada: é o desvio sistemático da previsão média em relação à resposta média que desejamos estimar. Uma família de retas usada para aproximar uma relação fortemente curva pode apresentar viés elevado, pois suas restrições impedem representar adequadamente essa relação.

A **variância** mede a dispersão das previsões em torno de sua própria média:

$$\text{Var}_{\mathcal{D}}\big[\hat{h}_{\mathcal{D}}(x)\big] = \mathbb{E}_{\mathcal{D}}\left[\Big(\hat{h}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}\big[\hat{h}_{\mathcal{D}}(x)\big]\Big)^2\right].$$

No exemplo da máquina, há alta variância quando diferentes amostras de medições, coletadas pelo mesmo processo, produzem previsões muito diferentes para o mesmo tempo de funcionamento. Essa variabilidade das previsões ajustadas é distinta da variabilidade das próprias medições de consumo: mesmo sob condições semelhantes, o consumo observado pode flutuar por fatores não registrados na entrada.

Assim como em Estatística, um procedimento pode apresentar pequeno viés e alta variância, ou maior viés e menor variância. Em geral, não conhecemos $f(x)$ nem dispomos de muitas amostras independentes para calcular diretamente essas quantidades; elas servem como instrumentos conceituais para compreender o aprendizado. Os erros de treino e validação ajudam no diagnóstico, mas não são, respectivamente, medidas de viés e variância. A decomposição do erro quadrático em viés ao quadrado, variância e ruído é apresentada no Apêndice B.

Quando o modelo se ajusta excessivamente às particularidades da amostra de treinamento, reduzir ainda mais o custo nesses exemplos pode prejudicar as previsões em dados novos. Nessa situação, interessa limitar a sensibilidade do modelo à amostra, mesmo que isso implique aceitar um erro de treinamento maior. Uma maneira de controlar esse ajuste é a *regularização*, que introduz restrições ou preferências no aprendizado. Em vez de considerar apenas o custo nos exemplos, podemos também penalizar determinadas características dos parâmetros. Uma forma comum é acrescentar uma penalidade quadrática sobre os pesos:

$$J_{\text{reg}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \lambda \|\mathbf{w}\|_2^2, \qquad \lambda \geq 0,$$

onde $\mathbf{w}$ reúne os pesos penalizados e $\|\mathbf{w}\|_2^2 = \sum_j w_j^2$. O custo J continua medindo o ajuste aos dados; o termo adicional favorece soluções com pesos de menor magnitude.

No modelo $h_{w,b}(x) = wx + b$, se penalizarmos apenas w, teremos $J_{\text{reg}}(w, b) = J(w, b) + \lambda w^2$. Para o candidato $w = 1{,}5$, $b = 0{,}5$, cujo MSE de treinamento calculado anteriormente foi 0,0175, a escolha ilustrativa $\lambda = 0{,}01$ produziria

$$J_{\text{reg}} = 0{,}0175 + 0{,}01(1{,}5)^2 = 0{,}04.$$

O valor 0,04 é o objetivo regularizado desse candidato, não seu novo MSE. Ao treinar com esse objetivo, o algoritmo poderá escolher outros parâmetros para equilibrar ajuste e penalização. Como a penalidade depende da escala das entradas e dos pesos, seu efeito também depende da representação adotada.

O hiperparâmetro λ controla esse equilíbrio e deve ser escolhido com auxílio da validação. Uma penalidade pequena pode ser insuficiente para controlar o ajuste excessivo; uma penalidade grande pode impedir que o modelo capture padrões relevantes. A regularização não garante melhora: seu efeito precisa ser avaliado em dados separados do treinamento.

3.4 Perda de treinamento e qualidade de uso

Até aqui, comparamos modelos principalmente por uma média de perdas. Essa medida é útil, mas não descreve todas as consequências de usar o modelo. No exemplo da máquina, dois modelos podem apresentar o mesmo MSE e cometer erros diferentes: um pode superestimar o consumo com frequência, enquanto outro o subestima. Se a previsão for usada para reservar energia, subestimar o consumo pode ter consequências distintas de superestimá-lo.

Por isso, além da média, devemos examinar a direção, a magnitude e as condições dos erros. Um MSE baixo no conjunto completo pode esconder desempenho ruim sob carga pesada ou em tempos de funcionamento menos frequentes. O critério de avaliação deve refletir o uso pretendido, e os dados avaliados devem representar as situações relevantes.

Em classificação, também precisamos observar quais erros o modelo comete, e não apenas quantas decisões estão corretas. Considere um conjunto de mil transações, das quais dez são fraudulentas. Um modelo que classifica todas como legítimas acerta 990 casos, mas deixa passar todas as fraudes. Embora a quantidade de acertos pareça elevada, o modelo não cumpre a finalidade de detectar fraudes. Para avaliar sua utilidade, precisamos distinguir fraudes não detectadas de transações legítimas indevidamente sinalizadas e considerar as consequências de cada erro. Métricas que ajudam nessa análise são apresentadas no Apêndice B.5.

A função de perda fornece um objetivo para o treinamento, enquanto a avaliação investiga se o comportamento aprendido atende à aplicação. Essa distinção reaparecerá em modelos de linguagem: reduzir a perda de predição de tokens e produzir respostas úteis, verdadeiras ou seguras são objetivos relacionados, mas uma melhora no primeiro não garante uma melhora em todos os demais.

Pergunta 4

Considere os três modelos de previsão de consumo comparados anteriormente: A, com MSE de treino/validação de 1,20/1,25; B, com 0,03/0,05; e C, com 0,001/0,90. **(a)** Escolha provisoriamente um modelo e justifique a decisão, indicando uma informação adicional necessária antes de aprová-lo para uso. **(b)** Para a diferença entre os erros de treino e validação do modelo C, proponha duas explicações distintas e descreva uma investigação que ajude a diferenciá-las. **(c)** Um colega sugere testar vários valores de λ e escolher aquele que produzir o menor erro no conjunto de teste. Explique o problema dessa proposta e descreva como realizar a escolha preservando uma avaliação final separada.

4 Modelos Lineares e Aprendizado por Gradiente

Nas seções anteriores, comparamos funções que preveem o consumo de energia de uma máquina e discutimos como avaliar suas previsões. Entretanto, ainda não explicamos como os parâmetros dessas funções podem ser encontrados a partir dos dados. Nesta seção, estudaremos um procedimento de ajuste, o gradiente descendente, começando por modelos lineares. Depois, adaptaremos a saída do modelo e a função de perda para tarefas de classificação, preparando o estudo de redes neurais e modelos de linguagem.

4.1 Regressão linear e erro quadrático

No exemplo da máquina, usamos o modelo $h_{w,b}(x) = wx+b$, em que x representa o tempo de funcionamento e a saída estima o consumo de energia. O peso w controla quanto a previsão varia quando o tempo aumenta, enquanto b determina a previsão para $x = 0$. O treinamento ajusta esses dois parâmetros a partir das medições disponíveis.

Também discutimos que o tempo pode ser insuficiente para explicar o consumo quando a máquina opera sob cargas diferentes. Se representarmos o tempo por x_1 e uma medida da carga de operação por x_2, podemos ampliar o modelo:

$$\hat{y} = w_1 x_1 + w_2 x_2 + b.$$

Agora, cada característica tem um peso próprio. Mantendo a carga fixa, uma unidade adicional de tempo altera a previsão em w_1; mantendo o tempo fixo, uma unidade adicional de carga altera a previsão em w_2. Esse modelo pressupõe contribuições aditivas das características, uma escolha que deve ser avaliada diante dos dados.

Com d características, escrevemos

$$\hat{y} = h_{\mathbf{w},b}(\mathbf{x}) = \sum_{j=1}^{d} w_j x_j + b = \mathbf{w}^\top \mathbf{x} + b,$$

onde $\mathbf{x} = (x_1, \ldots, x_d)^\top$ reúne as características e $\mathbf{w} = (w_1, \ldots, w_d)^\top$ reúne os pesos. O produto $\mathbf{w}^\top \mathbf{x}$ é apenas uma forma compacta de escrever a soma ponderada.

Esse é o modelo de *regressão linear*. Com a presença de b, a função é, rigorosamente, afim, mas a denominação "modelo linear" é usual. O parâmetro b também é chamado de *viés* ou *intercepto*; esse nome não deve ser confundido com o viés estatístico discutido na seção anterior.

Para ajustar os parâmetros, retomamos o mesmo erro quadrático médio já utilizado:

$$J(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)} \right)^2 .$$

A diferença é que agora a previsão pode depender de várias características. A tarefa de treinamento continua sendo procurar valores dos parâmetros que reduzam esse custo nos dados de treinamento. A solução analítica para regressão linear e exercícios adicionais ficam no Apêndice A; aqui, estudaremos um procedimento iterativo que também poderá ser empregado em outros modelos.

4.2 Gradiente descendente

Suponha que tenhamos escolhido valores iniciais para os parâmetros e calculado o custo correspondente. Como decidir que alteração fazer? Uma pergunta útil é: se aumentarmos um pouco um peso, mantendo os demais parâmetros fixos, o custo aumenta ou diminui?

A *derivada parcial* do custo em relação a um parâmetro descreve essa variação local. Se $\partial J/\partial w_j$ for positiva, um pequeno aumento de w_j tende a aumentar o custo; para reduzi-lo, podemos diminuir esse peso. Se a derivada for negativa, um pequeno aumento de w_j tende a diminuir o custo. Essa interpretação é local: ela não garante que uma alteração arbitrariamente grande tenha o mesmo efeito.

O *gradiente* reúne as derivadas parciais em um vetor. Para o modelo com uma única característica e parâmetros $\boldsymbol{\theta} = (w, b)$, temos

$$\nabla_{\boldsymbol{\theta}} J = \begin{bmatrix} \frac{\partial J}{\partial w} \\ \frac{\partial J}{\partial b} \end{bmatrix} .$$

Esse vetor indica a direção de maior crescimento local do custo. O *gradiente descendente* modifica os parâmetros na direção oposta:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}), \qquad \alpha > 0.$$

A *taxa de aprendizado* α controla o tamanho da atualização. Uma taxa pequena pode tornar o treinamento lento; uma taxa grande pode ultrapassar uma região de menor custo, aumentar o erro ou causar divergência. A direção indica uma possibilidade de redução local, mas o tamanho do passo também importa.

Para regressão linear com MSE, definindo o erro de cada exemplo como $e_i = \hat{y}^{(i)} - y^{(i)}$, as derivadas são

$$\frac{\partial J}{\partial w_j} = \frac{2}{n} \sum_{i=1}^{n} e_i x_j^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{2}{n} \sum_{i=1}^{n} e_i .$$

O fator 2 vem da derivada do quadrado do erro. A derivada em relação a cada peso também depende do valor da característica correspondente. Todos os componentes do gradiente são calculados com os parâmetros atuais; só depois ocorre a atualização conjunta.

Exemplo resolvido: melhorando o candidato anterior. Retome as quatro medições da máquina e o candidato $h(x) = 1{,}5x + 0{,}5$. Seu MSE de treinamento, calculado anteriormente, é 0,0175. Podemos verificar se uma atualização por gradiente melhora esse candidato.

Tempo x (h)	Consumo y (kWh)	Previsão $\hat{y}$ (kWh)	Erro $e = \hat{y} - y$
1	2,1	2,0	−0,1
2	3,3	3,5	0,2
3	5,1	5,0	−0,1
4	6,4	6,5	0,1

Para o peso w, obtemos

$$\begin{aligned}\frac{\partial J}{\partial w} &= \frac{2}{4}\left[(-0{,}1)1 + (0{,}2)2 + (-0{,}1)3 + (0{,}1)4\right] \\ &= \frac{1}{2}\left[-0{,}1 + 0{,}4 - 0{,}3 + 0{,}4\right] = 0{,}20.\end{aligned}$$

Para o intercepto b,

$$\frac{\partial J}{\partial b} = \frac{2}{4}\left[-0{,}1 + 0{,}2 - 0{,}1 + 0{,}1\right] = 0{,}05.$$

Com $\alpha = 0{,}1$, atualizamos os dois parâmetros:

$$w' = 1{,}5 - 0{,}1(0{,}20) = 1{,}48, \qquad b' = 0{,}5 - 0{,}1(0{,}05) = 0{,}495.$$

A nova função é $h'(x) = 1{,}48x + 0{,}495$. Suas previsões para os quatro tempos são 1,975, 3,455, 4,935 e 6,415 kWh. Portanto,

$$\begin{aligned}J' &= \frac{(-0{,}125)^2 + (0{,}155)^2 + (-0{,}165)^2 + (0{,}015)^2}{4} \\ &= 0{,}016775.\end{aligned}$$

O custo caiu de 0,0175 para 0,016775. O candidato anterior já representava bem as medições, mas ainda podia ser melhorado segundo o critério de treinamento. Essa redução não demonstra, por si só, melhora nas previsões de validação ou teste.

Uma atualização é apenas uma etapa do treinamento. O procedimento repete o seguinte ciclo:

1. Calcular as previsões com os parâmetros atuais.
2. Calcular o custo e suas derivadas em relação aos parâmetros.
3. Atualizar os parâmetros usando a taxa de aprendizado.
4. Repetir até atingir um critério de parada, como um número máximo de atualizações ou uma variação suficientemente pequena do custo.

O gradiente deve ser recalculado após cada atualização, pois as previsões, os erros e a inclinação local da função de custo podem ter mudado. Ao final, a validação e o teste continuam cumprindo os papéis discutidos na seção anterior.

Conexão com a aproximação de funções no AR. No Q-learning com aproximação linear, já encontramos uma atualização semelhante. Para $Q_{\mathbf{w}}(s,a) = \mathbf{w}^\top \mathbf{f}(s,a)$, usamos

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{f}(s,a),$$

onde δ é o alvo TD menos a previsão atual. A estrutura é semelhante à atualização de regressão: uma diferença entre alvo e previsão orienta a alteração dos pesos, levando em conta as características da entrada. A diferença de sinal em relação à escrita com e_i decorre da convenção: aqui definimos e_i como previsão menos alvo, enquanto δ é alvo menos previsão.

No caso do Q-learning, o alvo é tratado como constante durante essa atualização, embora possa depender dos próprios parâmetros do modelo. Por isso, essa regra é chamada de *semigradiente*. Essa ressalva distingue a atualização usual de Q-learning do gradiente completo de um objetivo que inclua a dependência do alvo em relação aos parâmetros.

4.3 Treinamento em lote, SGD e mini-lotes

No exemplo anterior, usamos todas as quatro medições para calcular o gradiente antes de atualizar os parâmetros. Esse procedimento é chamado de gradiente descendente *em lote*. Com poucos exemplos, o cálculo é simples; com milhões de exemplos, percorrer todo o conjunto antes de cada atualização pode ser caro.

Uma alternativa é usar menos exemplos por atualização. No *gradiente descendente estocástico* (SGD, do inglês *stochastic gradient descent*), usamos um exemplo sorteado por passo. Na versão em *mini-lotes*, usamos um subconjunto de exemplos e calculamos a média de suas contribuições para o gradiente. Essa versão é muito usada no treinamento de redes neurais.

No exemplo da máquina, se usarmos apenas a primeira medição, cujo erro no candidato anterior era $-0{,}1$, as derivadas estimadas serão

$$\frac{\partial \ell_1}{\partial w} = 2(-0{,}1)1 = -0{,}2, \qquad \frac{\partial \ell_1}{\partial b} = 2(-0{,}1) = -0{,}2.$$

Esses valores diferem do gradiente calculado com os quatro exemplos e, nesse caso, até apontam para alterações de sinal contrário. Não há contradição: melhorar a previsão de um exemplo não implica melhorar imediatamente a média sobre todos os exemplos.

Com amostragem adequada, o gradiente de um exemplo ou mini-lote fornece uma estimativa do gradiente médio de treinamento. As atualizações podem ser mais baratas, mas também mais ruidosas. Por isso, o custo no conjunto completo pode oscilar durante o treinamento; não se exige que cada passo o reduza.

Uma *atualização* é uma modificação dos parâmetros. Uma *época* corresponde a uma passagem pelo conjunto de treinamento, no esquema usual em que percorremos seus exemplos, frequentemente em uma ordem embaralhada. Se houver cem exemplos e cada mini-lote contiver dez, uma época terá dez atualizações. O tamanho do mini-lote e a taxa de aprendizado são hiperparâmetros do procedimento de treinamento.

4.4 Escala das entradas e padronização

Além da quantidade de exemplos usada por atualização, a forma numérica das entradas pode afetar a otimização. As derivadas dos pesos incluem os valores das características;

por isso, escalas muito diferentes podem produzir sensibilidades muito diferentes do custo em relação aos parâmetros.

Imagine que um modelo receba o tempo de funcionamento em horas, com valores próximos de unidades, e a potência de operação em watts, com valores próximos de milhares. Essa diferença de escala pode dificultar a escolha de uma única taxa de aprendizado adequada a todos os pesos, tornando a otimização lenta ou instável.

Uma transformação comum é a *padronização*: para cada característica j, subtraímos sua média μ_j e dividimos por seu desvio padrão s_j:

$$\widetilde{x}_j = \frac{x_j - \mu_j}{s_j}.$$

Quando $s_j > 0$, a característica padronizada tem média zero e variância um no conjunto usado para calcular essas estatísticas. Características constantes exigem tratamento separado, pois seu desvio padrão é zero. A padronização muda a representação numérica; ela não acrescenta novas informações às entradas.

Separação dos dados e vazamento de informação. As médias e os desvios padrão usados nessa transformação devem ser calculados apenas no conjunto de treinamento. Os mesmos valores são então aplicados à validação, ao teste e às novas entradas recebidas pelo modelo. Não recalculamos a transformação separadamente em cada um desses conjuntos.

Se utilizarmos também os dados de teste para calcular as estatísticas de padronização, a representação usada no treinamento passará a depender de informação que deveria permanecer reservada à avaliação. Esse é um exemplo de *vazamento de dados*. Mesmo sem usar os alvos de teste, estamos permitindo que dados reservados influenciem o desenvolvimento. Separar os conjuntos antes de ajustar as transformações ajuda a preservar o papel de cada subconjunto.

4.5 Regressão logística: do escore à probabilidade

Até aqui, a saída do modelo representa um valor numérico, como consumo de energia. Considere agora a tarefa de estimar a probabilidade de um e-mail ser spam. Uma combinação linear pode produzir qualquer número real, inclusive valores negativos ou maiores que um. Portanto, não podemos interpretá-la diretamente como uma probabilidade.

Na *regressão logística*, começamos calculando um *escore*:

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

Em seguida, aplicamos a função *sigmoide*, que transforma esse escore em um valor entre zero e um:

$$\hat{p} = P_{\boldsymbol{\theta}}(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

No exemplo de spam, convencionamos $y = 1$ para spam e $y = 0$ para mensagem legítima. A saída $\hat{p}$ estima a probabilidade da classe positiva; a probabilidade estimada da outra classe é $1 - \hat{p}$. Essas probabilidades são estimativas produzidas pelo modelo, cuja adequação precisa ser verificada com dados separados do treinamento.

Exemplo: do escore à estimativa probabilística. Se uma mensagem recebe escore $z = 0$, então

$$\hat{p} = \frac{1}{1 + e^{0}} = \frac{1}{2} = 0{,}5.$$

Se outra mensagem recebe escore $z = \log 3$, usando o logaritmo natural, então

$$\hat{p} = \frac{1}{1 + e^{-\log 3}} = \frac{1}{1 + \frac{1}{3}} = \frac{3}{4} = 0{,}75.$$

Escores positivos produzem probabilidades superiores a 0,5; escores negativos produzem probabilidades inferiores a 0,5.

Estimar uma probabilidade e escolher uma classe são operações distintas. Uma regra simples prevê spam quando $\hat{p} \geq 0{,}5$, o que equivale a $z \geq 0$. Esse limiar pode ser alterado conforme as consequências de bloquear uma mensagem legítima ou deixar passar spam. Apesar da transformação não linear da sigmoide, a fronteira de decisão com um limiar fixo continua sendo linear no espaço das características.

Para treinar o modelo, usaremos a *entropia cruzada binária*:

$$\ell(\hat{p}, y) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

Se $y = 1$, a perda se reduz a $-\log \hat{p}$; se $y = 0$, vale $-\log(1 - \hat{p})$. Nos dois casos, ela penaliza o modelo por atribuir pouca probabilidade à classe observada.

Para a mensagem com $\hat{p} = 0{,}75$, se o alvo for spam, a perda será

$$\ell(0{,}75, 1) = -\log(0{,}75) \approx 0{,}288.$$

Se o alvo for mensagem legítima, a perda será

$$\ell(0{,}75, 0) = -\log(0{,}25) \approx 1{,}386.$$

A previsão é a mesma, mas a avaliação muda conforme a classe observada. O treinamento minimiza a média dessas perdas nos exemplos, ajustando $\mathbf{w}$ e b por um procedimento como o gradiente descendente. Apesar do nome, regressão logística é um modelo de classificação.

4.6 Classificação multiclasse: logits, softmax e entropia cruzada

A regressão logística apresentada acima distingue duas classes. Como representar uma distribuição quando há mais alternativas? Suponha que desejemos classificar uma mensagem em três categorias mutuamente exclusivas: pessoal, trabalho ou spam.

Em um classificador linear multiclasse, cada classe k tem seu próprio vetor de pesos e seu próprio intercepto:

$$z_k = \mathbf{w}_k^\top \mathbf{x} + b_k, \qquad k = 1, \ldots, K.$$

Para uma mesma entrada, calculamos assim um vetor de escores $\mathbf{z} = (z_1, \ldots, z_K)$, chamados *logits*. Cada escore indica o favorecimento de uma classe pelo modelo, mas ainda não é uma probabilidade. Os logits podem ser positivos, negativos ou nulos.

A função *softmax* transforma esses escores em uma distribuição:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \sum_{k=1}^{K} p_k = 1.$$

O numerador é positivo, e o denominador soma as contribuições de todas as classes. Assim, cada probabilidade depende da comparação entre os logits. Dentro de uma mesma distribuição, um logit maior corresponde a uma probabilidade maior.

Se precisarmos escolher uma única classe, podemos selecionar aquela com maior probabilidade. Entretanto, a distribuição contém mais informação do que essa decisão: ela permite distinguir uma escolha com alternativas quase empatadas de outra em que uma classe recebe a maior parte da probabilidade.

Exemplo resolvido: probabilidades e perda. Para as classes pessoal, trabalho e spam, nessa ordem, suponha

$$\mathbf{z} = (\log 2, 0, 0).$$

Como $e^{\log 2} = 2$ e $e^0 = 1$, obtemos

$$\mathbf{p} = \left(\frac{2}{2+1+1}, \frac{1}{2+1+1}, \frac{1}{2+1+1} \right) = (0{,}50, 0{,}25, 0{,}25).$$

A classe mais provável é pessoal, mas as demais continuam recebendo probabilidades positivas.

Se a classe observada é y, a entropia cruzada para esse exemplo é

$$\ell(\mathbf{p}, y) = -\log p_y,$$

onde p_y é a probabilidade atribuída à classe observada. Usamos logaritmos naturais. Se a mensagem for pessoal, a perda será

$$\ell = -\log(0{,}50) \approx 0{,}693.$$

Se for uma mensagem de trabalho, a perda será

$$\ell = -\log(0{,}25) \approx 1{,}386.$$

Novamente, o critério penaliza mais a previsão que atribui menor probabilidade à classe observada.

A entropia cruzada binária e essa expressão multiclasse seguem o mesmo princípio: considerar o negativo do logaritmo da probabilidade atribuída ao alvo. Para treinar com muitos exemplos, calculamos a média das perdas e usamos seus gradientes para ajustar os parâmetros que produzem os logits. Mudam a forma da saída e a função de perda, mas permanece o mecanismo de aprendizado: prever, medir a discrepância e atualizar parâmetros.

Ponte para modelos de linguagem. Na predição do próximo token, cada token do vocabulário é uma classe possível. O contexto fornece a entrada, o modelo produz logits para os tokens, a softmax define uma distribuição e o token observado fornece o alvo da entropia cruzada. Como esse alvo é extraído do próprio texto, a tarefa pode ser construída de forma autossupervisionada. Um modelo de linguagem empregará uma arquitetura muito mais expressiva do que o classificador linear apresentado aqui. Ainda assim, o mecanismo de saída e perda será reutilizado. Na próxima seção, descrevemos como redes neurais aprendem representações intermediárias antes de produzir essas previsões.

Pergunta 5

Após uma atualização dos parâmetros do modelo de consumo, um aluno propõe reutilizar os erros calculados antes da atualização para executar o próximo passo. **(a)** Explique por que esses valores podem não representar mais o gradiente do custo nos parâmetros atuais. **(b)** Outro aluno afirma que uma atualização que melhora a previsão de uma medição deve reduzir o MSE de todo o conjunto. Construa um exemplo simples ou uma explicação que mostre por que essa conclusão não é garantida. **(c)** Ao adaptar o modelo para estimar a probabilidade de spam, seria suficiente interpretar diretamente a saída $\mathbf{w}^\top \mathbf{x} + b$ como uma probabilidade? Explique o papel da sigmoide e por que a perda deve avaliar a probabilidade atribuída à classe observada.

5 Redes Neurais e Representações Aprendidas

Na seção anterior, estudamos modelos que combinam diretamente as características de entrada para produzir uma previsão numérica ou escores de classificação. No exemplo da máquina, escolhemos características como tempo de funcionamento e carga de operação. Na classificação de e-mails, podemos escolher frequências de palavras e quantidade de links. Em ambos os casos, a representação fornecida ao modelo influencia os padrões que ele consegue aprender.

Em problemas mais complexos, pode ser difícil definir manualmente características que tornem a tarefa simples para um modelo linear. Redes neurais permitem introduzir transformações intermediárias entre a entrada e a saída e ajustar essas transformações a partir dos dados. Nesta seção, estudaremos como isso amplia a capacidade de representação do modelo e como podemos treinar seus parâmetros usando os mesmos elementos já apresentados: função de perda, cálculo de gradientes e atualização dos pesos.

5.1 De um neurônio a um MLP

Um *neurônio artificial* realiza duas operações: calcula uma combinação ponderada das entradas e aplica uma função de ativação ao resultado. Podemos escrevê-lo como

$$u = \mathbf{w}^\top \mathbf{x} + b, \qquad a = \phi(u),$$

onde $\mathbf{w}$ reúne os pesos, b é o intercepto e ϕ é a função de ativação. O valor a é a saída do neurônio.

Essa estrutura já apareceu na regressão logística. Naquele caso, aplicamos a sigmoide a uma combinação linear e interpretamos a saída como a probabilidade da classe positiva. Portanto, um classificador logístico binário pode ser representado por um único neurônio com ativação sigmoide. Essa equivalência ajuda a compreender a continuidade entre os modelos anteriores e as redes neurais.

O próximo passo é combinar vários neurônios. Em uma *camada*, diferentes neurônios recebem a mesma entrada, mas cada um possui seus próprios pesos e intercepto. Assim, cada neurônio pode produzir uma combinação diferente das características. Reunindo suas saídas em um vetor, obtemos

$$\mathbf{h} = \phi(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}),$$

onde cada linha de $W^{(1)}$ contém os pesos de um neurônio, $\mathbf{b}^{(1)}$ reúne os interceptos e ϕ é aplicada componente a componente.

O vetor $\mathbf{h}$ passa a ser uma nova representação da entrada. Uma camada seguinte pode combinar seus componentes para produzir os logits de classificação:

$$\mathbf{z} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)}, \qquad \mathbf{p} = \mathrm{softmax}(\mathbf{z}).$$

A softmax e a entropia cruzada cumprem os mesmos papéis da seção anterior. A novidade está no cálculo dos logits: em vez de combinar diretamente as características originais, a camada de saída combina uma representação intermediária que também depende de parâmetros aprendidos.

Essa organização é um *perceptron multicamadas* (MLP, do inglês *multilayer perceptron*). A camada intermediária é chamada de *oculta* porque seus valores não são fornecidos como alvos nos exemplos de treinamento: eles são calculados pela rede. Podemos adicionar outras camadas ocultas, fazendo com que a representação produzida por uma seja a entrada da seguinte.

A arquitetura especifica quantas camadas existem, quantos neurônios há em cada uma e como estão conectados. O treinamento ajusta os valores dos pesos e interceptos dessas conexões. Os círculos da camada de entrada na Figura 2 representam características fornecidas à rede, não neurônios que calculam novas transformações.

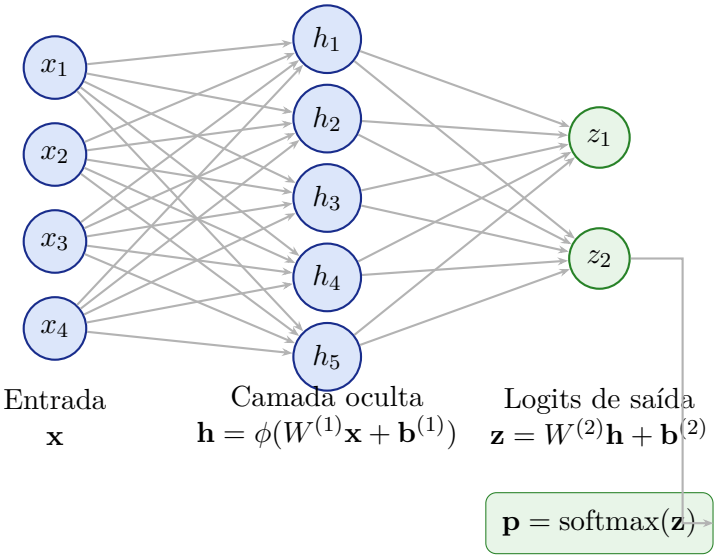


Figura 2: MLP com quatro características de entrada, cinco neurônios ocultos e dois logits de saída. Cada conexão possui um peso aprendido; os interceptos foram omitidos do desenho. A camada oculta transforma a entrada, a camada de saída produz logits e a softmax converte o vetor de logits em probabilidades.

Para a arquitetura da figura, $\mathbf{x} \in \mathbb{R}^4$, $\mathbf{h} \in \mathbb{R}^5$ e $\mathbf{z} \in \mathbb{R}^2$. Portanto, $W^{(1)}$ tem dimensão 5×4 e $W^{(2)}$ tem dimensão 2×5. Contando também os cinco interceptos ocultos e os dois de saída, há

$$5 \cdot 4 + 5 + 2 \cdot 5 + 2 = 37$$

parâmetros aprendidos.

5.2 Por que a não linearidade é necessária?

Adicionar camadas só amplia a capacidade de representação de certas maneiras se as operações entre elas forem adequadas. Suponha que retiremos a ativação da camada oculta. A transformação da entrada até os logits será

$$\begin{aligned}\mathbf{z} &= W^{(2)}(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \\ &= (W^{(2)}W^{(1)})\mathbf{x} + (W^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}) \\ &= \widetilde{W}\mathbf{x} + \widetilde{\mathbf{b}}.\end{aligned}$$

Assim, duas transformações afins consecutivas equivalem a uma única transformação afim. O mesmo raciocínio vale para mais camadas: sem ativações não lineares entre elas, os logits continuam sendo funções afins da entrada.

Uma ativação simples é a *ReLU* (*Rectified Linear Unit*):

$$\phi(u) = \max(0, u).$$

Ela devolve zero para entradas negativas e preserva entradas positivas. Por exemplo,

$$\mathrm{ReLU}(-2) = 0, \qquad \mathrm{ReLU}(3) = 3.$$

Ao aplicar essa operação a combinações diferentes das entradas, a rede pode produzir transformações com comportamentos distintos em diferentes regiões do espaço. Isso permite construir relações não lineares entre as características originais e os logits.

A ReLU da camada oculta e a softmax da saída cumprem funções diferentes. A primeira participa da construção da representação intermediária; a segunda converte os logits em uma distribuição sobre classes. A existência de uma softmax na saída, por si só, não substitui as ativações ocultas necessárias para ampliar a família de funções que produz os logits.

No exemplo da máquina, uma rede com ativações não lineares poderia representar mudanças na taxa de consumo ao longo do tempo. Para uma tarefa de regressão, a camada final pode produzir diretamente um valor numérico, sem softmax, e ser treinada com MSE. A saída e a perda devem ser escolhidas conforme a tarefa, assim como nos modelos anteriores. Outras ativações e dificuldades de treinamento são discutidas no Apêndice E.

5.3 Como uma camada oculta transforma o espaço de características

Uma maneira de compreender o papel das camadas ocultas é observar como elas modificam a disposição dos exemplos no espaço de características. Uma transformação não linear pode aproximar exemplos que devem receber a mesma classe e tornar mais simples a separação entre classes. Informalmente, dizemos que a rede pode "entortar" o espaço de entrada. Essa expressão é uma metáfora: a transformação pode comprimir ou sobrepor regiões, alterar distâncias e até mudar a dimensionalidade da representação.

Considere o problema XOR, com duas entradas binárias. Os exemplos $(0, 1)$ e $(1, 0)$ pertencem à classe positiva, enquanto $(0, 0)$ e $(1, 1)$ pertencem à classe negativa. No plano das entradas, as classes ocupam cantos alternados de um quadrado. Nenhuma reta consegue separar corretamente os quatro exemplos.

Agora considere uma camada oculta com dois neurônios ReLU:

$$h_1 = \mathrm{ReLU}(x_1 + x_2 - 1), \qquad h_2 = \mathrm{ReLU}(1 - x_1 - x_2).$$

Essa camada tem pesos e interceptos

$$W^{(1)} = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \qquad \mathbf{b}^{(1)} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Os valores foram escolhidos para ilustrar uma solução representável pela rede; não estamos afirmando que um procedimento de treinamento necessariamente encontrará esses mesmos parâmetros.

Aplicando a transformação aos quatro exemplos, obtemos:

Entrada (x_1, x_2)	Classe	Representação (h_1, h_2)
$(0, 0)$	$-$	$(0, 1)$
$(0, 1)$	$+$	$(0, 0)$
$(1, 0)$	$+$	$(0, 0)$
$(1, 1)$	$-$	$(1, 0)$

Os dois exemplos positivos passam a ocupar a mesma posição. Nessa nova representação, a reta $h_1 + h_2 = 0{,}5$ separa as classes, como mostra a Figura 3.

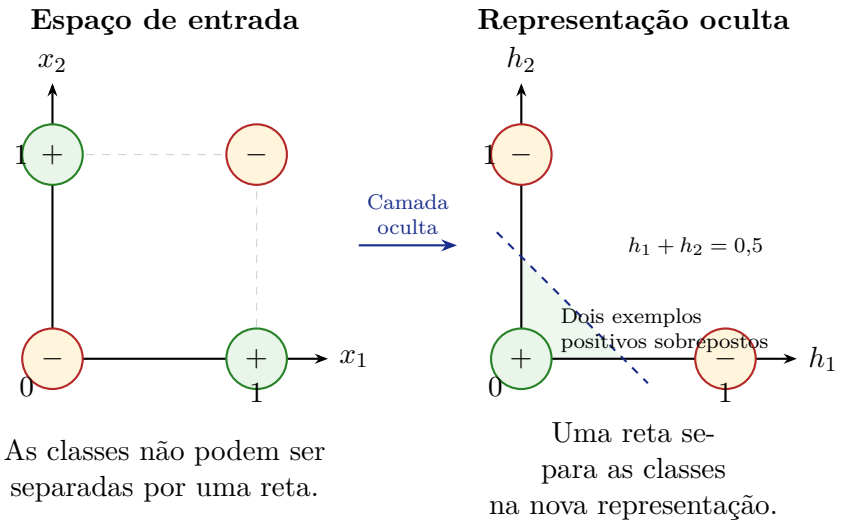


Figura 3: Transformação do problema XOR por uma camada oculta com dois neurônios ReLU. À esquerda, os exemplos não são linearmente separáveis. À direita, os dois exemplos positivos foram mapeados para $(0, 0)$, enquanto os negativos foram mapeados para $(0, 1)$ e $(1, 0)$. A fronteira tracejada é linear nas características ocultas, embora a decisão correspondente seja não linear nas entradas originais.

A camada de saída pode aproveitar essa representação calculando o escore $s = 0{,}5 - h_1 - h_2$ e escolhendo a classe positiva quando $s > 0$. Se usarmos dois logits, podemos definir $z_+ = s$ e $z_- = 0$ e aplicar softmax. Os exemplos positivos terão maior probabilidade para a classe positiva, enquanto os negativos terão maior probabilidade para a classe negativa.

Substituindo as expressões dos neurônios ocultos, a região classificada como positiva no espaço original é

$$0{,}5 < x_1 + x_2 < 1{,}5.$$

Portanto, a regra é linear em (h_1, h_2), mas não corresponde a uma única fronteira linear em (x_1, x_2). Esse é o ganho ilustrado pela camada oculta: transformar a entrada para que uma operação simples na saída possa resolver uma tarefa que antes exigia uma separação mais complexa.

Durante o treinamento, não fornecemos necessariamente as posições que cada exemplo deve ocupar na camada oculta. A perda da tarefa orienta o ajuste dos parâmetros que produzem essas posições. O exemplo mostra uma representação que resolve o XOR; em problemas reais, encontrar representações úteis depende dos dados, da arquitetura e do procedimento de otimização.

5.4 Exemplo resolvido: da entrada à perda

Para tornar o fluxo concreto, considere uma rede menor, com duas características de entrada, dois neurônios ocultos com ReLU e duas classes de saída. Os valores abaixo são ilustrativos e representam uma configuração dos parâmetros durante o treinamento:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \qquad W^{(1)} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \qquad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Primeiro, calculamos as combinações da camada oculta:

$$\mathbf{u} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)} = \begin{bmatrix} 1 \cdot 1 - 1 \cdot 2 \\ 1 \cdot 1 + 1 \cdot 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}.$$

Depois, aplicamos a ReLU a cada componente:

$$\mathbf{h} = \begin{bmatrix} \max(0, -1) \\ \max(0, 3) \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix}.$$

A entrada original $(1, 2)^\top$ foi transformada na representação $(0, 3)^\top$. Essa representação depende tanto da entrada quanto dos parâmetros da camada.

Suponha agora que

$$W^{(2)} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{b}^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Os logits serão

$$\mathbf{z} = W^{(2)}\mathbf{h} + \mathbf{b}^{(2)} = \begin{bmatrix} 3 \\ 0 \end{bmatrix}.$$

Aplicando softmax, obtemos

$$p_1 = \frac{e^3}{e^3 + e^0} \approx 0{,}953, \qquad p_2 = \frac{e^0}{e^3 + e^0} \approx 0{,}047.$$

Se o alvo observado for a primeira classe, a entropia cruzada será

$$\ell = -\log p_1 \approx 0{,}049.$$

Se o alvo for a segunda classe, a mesma previsão produzirá uma perda muito maior:

$$\ell = -\log p_2 \approx 3{,}049.$$

Até aqui, apenas calculamos o que a rede produz com os parâmetros atuais e avaliamos essa previsão. Para aprender, ainda precisamos determinar como os parâmetros afetam a perda e modificá-los.

5.5 Retropropagação e atualização dos parâmetros

Na regressão linear, escrevemos diretamente as derivadas do custo em relação aos pesos. Em uma rede neural, um peso da primeira camada afeta a representação oculta, que afeta os logits, que afetam as probabilidades e, finalmente, a perda. Precisamos calcular derivadas através dessa composição de operações.

A regra da cadeia fornece esse mecanismo. Para um caminho simples $w \to u \to a \to \ell$, temos

$$\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial a}\frac{\partial a}{\partial u}\frac{\partial u}{\partial w}.$$

Se um parâmetro influencia a perda por vários caminhos, somamos suas contribuições. A *retropropagação* (*backpropagation*) organiza esse cálculo de trás para frente, reutilizando derivadas intermediárias em vez de refazer os mesmos cálculos para cada parâmetro.

No exemplo anterior, podemos ilustrar uma dessas derivadas sem calcular o treinamento da rede inteira. Quando o alvo é a primeira classe, a combinação de softmax e entropia cruzada fornece

$$\frac{\partial \ell}{\partial z_1} = p_1 - 1 \approx -0{,}047.$$

Como $z_1 = W^{(2)}_{1,1}h_1 + W^{(2)}_{1,2}h_2 + b^{(2)}_1$ e $h_2 = 3$, a regra da cadeia produz

$$\frac{\partial \ell}{\partial W^{(2)}_{1,2}} = \frac{\partial \ell}{\partial z_1}\frac{\partial z_1}{\partial W^{(2)}_{1,2}} \approx (-0{,}047)\cdot 3 = -0{,}142.$$

O sinal negativo indica que um pequeno aumento desse peso, mantendo os demais parâmetros fixos, tende a reduzir a perda desse exemplo. Para alcançar os pesos da primeira camada, o cálculo continua através de $\mathbf{h}$ e da ativação. Na ReLU, a derivada é zero para entradas negativas e um para entradas positivas; no ponto zero, adota-se uma convenção computacional.

O treinamento reúne três operações distintas:

1. **Propagação para a frente (*forward*):** calcular representações, logits, previsões e perda com os parâmetros atuais.
2. **Retropropagação (*backward*):** calcular as derivadas da perda em relação aos parâmetros pela regra da cadeia.
3. **Atualização:** usar essas derivadas em um algoritmo de otimização, como SGD, para modificar os parâmetros.

Em um mini-lote, agregamos as perdas dos exemplos e calculamos o gradiente do custo correspondente. Depois, aplicamos uma atualização como

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J,$$

onde $\boldsymbol{\theta}$ agora reúne os pesos e interceptos de todas as camadas. Os gradientes são calculados com os parâmetros atuais antes da atualização conjunta, assim como no exemplo de regressão linear.

Retropropagação não é o otimizador.

A retropropagação calcula derivadas; o otimizador usa essas derivadas para escolher uma atualização. A função de perda define o que será penalizado. Portanto, calcular corretamente os gradientes e reduzir a perda não garante que o objetivo escolhido represente todas as exigências da aplicação. A distinção entre perda de treinamento e qualidade de uso continua válida.

5.6 Características projetadas e representações aprendidas

No Q-learning linear, escolhemos características $f_i(s, a)$ para representar estados e ações. Na classificação de spam, podemos escolher frequências de palavras e quantidade de links. Essas características são projetadas antes do ajuste dos pesos do modelo.

Em um MLP, a representação oculta $\mathbf{h}$ também depende de parâmetros ajustados durante o treinamento. Ao modificar $W^{(1)}$ e $\mathbf{b}^{(1)}$, alteramos como as entradas são representadas para a camada seguinte. A perda da tarefa orienta simultaneamente o aprendizado dessa representação e o aprendizado da regra de saída. Esse processo é chamado de *aprendizado de representações*.

No exemplo numérico, $\mathbf{h} = (0, 3)^\top$ era a representação de uma entrada para uma configuração específica dos parâmetros. Depois de atualizá-los, a mesma entrada poderá produzir outra representação. As camadas ocultas não recebem, nesse caso, uma lista de características que deveriam descobrir: suas transformações são ajustadas conforme contribuem para reduzir a perda final.

Isso não elimina todas as decisões sobre representação. Ainda precisamos escolher quais dados fornecer, como organizá-los e qual arquitetura usar. Também não há garantia de que cada componente oculto corresponda a um conceito humano: informações relevantes podem estar distribuídas entre várias coordenadas. A rede pode inclusive aprender atalhos, como associar uma característica incidental dos dados ao alvo, sem capturar a regularidade que desejávamos.

Embeddings e a próxima nota. Um *embedding* representa uma entidade por um vetor. No caso que estudaremos em modelos de linguagem, cada token terá um vetor aprendido, armazenado em uma linha de uma matriz de parâmetros. Durante o treinamento, a perda de predição orientará o ajuste desses vetores e dos demais pesos da rede. Assim, embeddings e representações ocultas participam da mesma ideia geral: construir representações úteis para uma tarefa por meio do aprendizado. Aprender esses vetores não exige executar previamente k-means ou PCA; o objetivo de predição pode fornecer o sinal para ajustá-los. A nota seguinte desenvolverá como tokens, embeddings e transformações da sequência se combinam em um modelo de linguagem.

5.7 Generalização e acompanhamento do treinamento

A capacidade de aprender representações amplia as funções que a rede pode produzir, mas não elimina o problema da generalização. A rede pode aprender regularidades úteis ou ajustar particularidades dos exemplos de treinamento. Continuamos precisando separar os dados, acompanhar a validação, controlar o ajuste e analisar os tipos de erro.

Considere duas etapas do treinamento de uma rede, avaliadas pela mesma perda média de entropia cruzada nos mesmos conjuntos:

Etapa	Treinamento	Validação
Anterior	0,30	0,35
Posterior	0,10	0,60

Na etapa posterior, a rede melhorou seu ajuste aos exemplos de treinamento, mas piorou nas previsões de validação. Essa evolução é um sinal de possível overfitting. Os valores precisam ser interpretados em relação à tarefa, mas a redução da perda de treino claramente não foi acompanhada de melhora na validação.

Podemos investigar a capacidade da arquitetura, a representatividade dos dados e a intensidade da regularização. A penalização dos pesos discutida anteriormente também pode ser aplicada às matrizes da rede. Mais dados representativos, mudanças na arquitetura ou uma penalidade diferente podem ajudar, dependendo da causa identificada. Não há uma regra de que redes maiores sempre generalizam pior: dados, arquitetura, otimização e regularização atuam em conjunto.

Parada antecipada. Como o treinamento ocorre ao longo de várias atualizações e épocas, não precisamos utilizar obrigatoriamente os parâmetros da última etapa executada. A *parada antecipada* (*early stopping*) acompanha a perda de validação e guarda os parâmetros de uma etapa com bom resultado. Se a validação deixar de melhorar por um período definido, podemos interromper o treinamento e recuperar os melhores parâmetros registrados.

No exemplo da tabela, os parâmetros da etapa anterior seriam preferíveis aos da posterior pelo critério de validação apresentado. Essa decisão é orientada pela validação, não pelo teste. A arquitetura, a penalização dos pesos e a regra de parada fazem parte das escolhas de desenvolvimento; o teste permanece reservado para avaliar o modelo selecionado.

Pergunta 6

Considere um MLP que classifica e-mails usando uma camada oculta com ReLU e uma saída softmax. **(a)** Um aluno afirma que retirar a ReLU não limita a rede, pois ela continua tendo duas matrizes de pesos e uma softmax. Você concorda? Justifique examinando a transformação entre a entrada e os logits. **(b)** Outro aluno propõe treinar apenas a camada de saída, mantendo os pesos e interceptos ocultos fixos. Nesse procedimento, a representação oculta de uma mesma entrada muda durante o treinamento? Compare com o ajuste de todas as camadas e explique o papel da retropropagação nessa diferença. **(c)** Durante o treinamento completo, a perda de treino diminui enquanto a de validação aumenta. Isso prova que as representações ficaram melhores para exemplos novos? Justifique e descreva como escolheria os parâmetros finais sem consultar o teste.

6 Síntese e Preparação para Modelos de Linguagem

O mecanismo comum aos modelos estudados é: representar uma entrada, produzir uma previsão, medir uma perda e ajustar parâmetros. Generalização exige avaliar o que ocorre em exemplos separados do treinamento. A arquitetura restringe as funções representáveis; dados e objetivo determinam que sinais orientarão o ajuste.

Fundamento desta nota	Reutilização na nota seguinte
Entrada e alvo de predição	Prefixo de texto e próximo token observado
Parâmetros e representações	Embeddings e pesos das camadas do modelo
Logits, softmax e entropia cruzada	Distribuição e perda de predição do próximo token
Retropropagação e mini-lotes	Ajuste de muitos parâmetros a partir de sequências
Treino, validação e generalização	Avaliação em textos separados e em condições de uso
Perda e critério de qualidade	Diferença entre prever texto e responder adequadamente

O que levar para a próxima leitura. Você deve conseguir explicar como dados geram alvos, como a perda orienta o treinamento e por que uma distribuição de probabilidades pode ser a saída de uma rede. Tokens, informação posicional, atenção e Transformers serão desenvolvidos na nota sobre Modelos de Linguagem. O conhecimento de AR será retomado posteriormente na discussão de alinhamento por preferências.

7 Atividades Intra-Classe do Núcleo

As atividades abaixo são organizadas pelos três encontros. Resolva-as em duplas ou pequenos grupos, explicitando o raciocínio. As atividades dos apêndices são opcionais e não constituem pré-requisitos para estas.

7.1 Encontro 1: tarefas, dados e avaliação

1. **Identificação do problema.** Para classificar spam e para prever o próximo token, identifique entrada, alvo e origem do sinal de aprendizado. Qual tarefa exige rótulos externos e qual pode extrair seus alvos dos próprios dados?

2. **Vazamento.** Um conjunto contém várias mensagens do mesmo encadeamento de e-mails. A equipe divide mensagens aleatoriamente e ajusta a normalização em todo o conjunto. Identifique dois riscos e proponha um procedimento de avaliação compatível com o uso em encadeamentos novos.

3. **Diagnóstico.** Três modelos têm MSE de treino/validação iguais a 120/125, 45/52 e 2/380. Qual apresenta evidência de overfitting? Qual selecionaria inicialmente? Que referência adicional seria necessária antes de afirmar que 120/125 representa underfitting?

4. **Qualidade da métrica.** Em 1.000 mensagens, 990 são legítimas. Um modelo prevê todas como legítimas. Calcule a acurácia e explique o que esse resultado não revela. Qual tipo de erro o modelo comete nas dez mensagens restantes?

7.2 Encontro 2: aprender parâmetros e prever probabilidades

1. **Um passo de gradiente.** Use os dados $(1, 3)$ e $(2, 5)$, o modelo $h(x) = wx + b$, parâmetros iniciais $w = 1$, $b = 1$ e taxa $\alpha = 0{,}1$. Calcule previsões, MSE, gradientes,

parâmetros atualizados e novo MSE. Explique por que todos os gradientes usam os parâmetros anteriores à atualização.

2. **Probabilidade binária.** Para uma mensagem com alvo $y = 1$, compare as perdas de modelos que atribuem probabilidade 0,9 e 0,1 à classe positiva. Ambos acertariam com limiar 0,5? O que a perda informa além da decisão?

3. **Softmax.** Calcule a distribuição correspondente a $\mathbf{z} = (0, \log 3, 0)$. Calcule a entropia cruzada se o alvo for a segunda classe e se for a primeira. O que acontece com a distribuição se somarmos a mesma constante a todos os logits?

4. **Conexão com AR.** Compare o alvo supervisionado com o alvo $r+\gamma \max_{a'} Q(s', a')$. Por que a dependência desse segundo alvo dos parâmetros exige cuidado ao interpretar a atualização como gradiente?

7.3 Encontro 3: redes, representações e integração

1. **Arquitetura.** Para uma entrada com 4 características, camada oculta com 5 neurônios e saída com 3 classes, indique as dimensões de $W^{(1)}$, $\mathbf{b}^{(1)}$, $W^{(2)}$ e $\mathbf{b}^{(2)}$. Conte os parâmetros aprendidos.

2. **Não linearidade.** Mostre algebricamente que duas camadas afins sem ativação equivalem a uma. Por que isso limita a rede, mesmo que aumentemos o número de camadas?

3. **Treinamento.** Ordene e explique: atualizar parâmetros; calcular logits; calcular a perda; produzir a representação oculta; calcular gradientes. Quais operações pertencem ao forward, ao backward e ao otimizador?

4. **Generalização.** Uma rede melhora continuamente no treino, mas sua perda de validação começa a subir após a época 8. Explique como usaria early stopping e como escolheria uma penalização dos pesos. Quando usaria o teste?

5. **Ponte para linguagem.** Para o texto “aprendizado por reforço”, suponha, apenas neste exercício, que cada palavra seja um token. Construa um par prefixo–próximo token e explique o papel dos embeddings, dos logits e da entropia cruzada. Por que esse treinamento não garante que uma resposta gerada seja verdadeira?

Pergunta 7

Sem consultar fórmulas, explique a um colega como um modelo aprende a partir de exemplos e como verificar se aprendeu algo útil para dados novos. Inclua: parâmetros, perda, gradiente, validação, representação e um exemplo de autossupervisão. Se uma dessas conexões não estiver clara, retome a seção correspondente antes de iniciar a nota sobre Modelos de Linguagem.

Apêndices: Consulta e Aprofundamento

Como usar os apêndices. Os tópicos a seguir foram deslocados do roteiro de aulas para preservar a visão ampla do aprendizado de máquina sem aumentar a carga obrigatória. Incluem explicações, figuras, exemplos e atividades de aprofundamento. Não é necessário dominar árvores, ensembles, k-means, PCA, a decomposição viés–variância ou as métricas adicionais para iniciar a nota sobre Modelos de Linguagem.

Consulte cada apêndice quando quiser ampliar um tema. As atividades opcionais estão reunidas no Apêndice H. As referências a seções indicam a organização desta versão da nota.

A Aprofundamento em Modelos Lineares e Características

A.1 Regressão linear

O modelo mais simples de AM supervisionado é a *regressão linear*. Dado um vetor de características $\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top$, o modelo prevê:

$$h_{\mathbf{w},b}(\mathbf{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \mathbf{w}^\top \mathbf{x} + b$$

onde $\mathbf{w} = [w_1, \ldots, w_d]^\top$ é o vetor de pesos e b é o *viés* (*bias*), um parâmetro escalar que desloca a previsão.

Convenção: absorvendo o viés. É conveniente absorver b no vetor de pesos adicionando uma componente constante $x_0 = 1$ ao vetor de características: $\mathbf{x} \leftarrow [1, x_1, x_2, \ldots, x_d]^\top$ e $\mathbf{w} \leftarrow [b, w_1, \ldots, w_d]^\top$. Com isso, $h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$, e toda análise é feita em termos de um único vetor de parâmetros $\mathbf{w} \in \mathbb{R}^{d+1}$. Essa convenção é padrão na literatura e simplifica a notação.

Para regressão linear, a função de perda padrão é o *erro quadrático médio* (*mean squared error*, MSE):

$$J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} \left(h_{\mathbf{w}}(\mathbf{x}^{(i)}) - y^{(i)} \right)^2 = \frac{1}{n} \sum_{i=1}^{n} \left(\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

Existe uma solução analítica fechada para o vetor ótimo de pesos, chamada *equação normal*:

$$\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y}$$

onde X é a matriz de dados ($n \times (d+1)$, uma linha por exemplo) e $\mathbf{y}$ é o vetor de rótulos.

Quando a equação normal falha. A equação normal requer inverter a matriz $X^\top X$, de tamanho $(d+1) \times (d+1)$. Isso tem custo $O(d^3)$, o que a torna impraticável para d grande (e.g., $d = 10^6$ em problemas de processamento de linguagem natural). Além disso, $X^\top X$ pode ser singular (não invertível) se as características forem linearmente dependentes. O gradiente descendente evita a inversão explícita. Em mínimos quadrados, fatorações QR, SVD ou a pseudoinversa também permitem tratar o problema; características dependentes podem tornar a solução não única.

A.1.1 Exemplo resolvido

Suponha um problema de regressão com apenas uma característica ($d = 1$): prever o preço de um imóvel (y, em mil reais) em função de sua área (x, em mš). Dados de treinamento:

Exemplo	x (área, mš)	y (preço, R$ mil)
1	50	200
2	100	350
3	150	500

Com $\mathbf{w} = [b, w_1]^\top$ e $x_0 = 1$, temos $h(x) = b + w_1 x$.

Passo 1: verificar um candidato. Suponha $b = 50$ e $w_1 = 3$, ou seja, $h(x) = 50 + 3x$.

$$\begin{aligned} h(50) &= 50 + 3 \times 50 = 200 \quad (\text{erro} = 0) \\ h(100) &= 50 + 3 \times 100 = 350 \quad (\text{erro} = 0) \\ h(150) &= 50 + 3 \times 150 = 500 \quad (\text{erro} = 0) \end{aligned}$$

Neste caso, os dados são exatamente lineares, então $J(\mathbf{w}) = 0$ é atingível.

Passo 2: verificar um candidato ruim. Suponha $b = 0$ e $w_1 = 4$:

$$\begin{aligned} J(\mathbf{w}) &= \frac{1}{3}\big[(200 - 200)^2 + (400 - 350)^2 + (600 - 500)^2\big] \\ &= \frac{1}{3}\big[0 + 2500 + 10000\big] = 4166{,}7 \end{aligned}$$

O MSE positivo indica que este não é o modelo ótimo.

A.2 Gradiente descendente

Para minimizar $J(\mathbf{w})$, o algoritmo mais geral é o *gradiente descendente*. A ideia é simples: a partir de um ponto inicial $\mathbf{w}^{(0)}$, repetidamente mova-se na direção oposta ao gradiente de J, a direção de maior descida local:

$$\mathbf{w} \;\leftarrow\; \mathbf{w} - \alpha\, \nabla_{\mathbf{w}} J(\mathbf{w})$$

onde $\alpha > 0$ é a *taxa de aprendizado* e $\nabla_{\mathbf{w}} J(\mathbf{w})$ é o gradiente de J em relação a $\mathbf{w}$.

Para regressão linear com MSE, o gradiente em relação ao peso w_j é:

$$\frac{\partial J}{\partial w_j} = \frac{2}{n} \sum_{i=1}^{n} \left(\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)}\right) x_j^{(i)}$$

A regra de atualização para cada peso w_j é, portanto:

$$w_j \;\leftarrow\; w_j - \alpha \cdot \frac{2}{n} \sum_{i=1}^{n} \underbrace{\left(\mathbf{w}^\top \mathbf{x}^{(i)} - y^{(i)}\right)}_{\text{erro na previsão}} x_j^{(i)}$$

Isso tem uma interpretação intuitiva: se o modelo está superestimando ($\hat{y} > y$) e a característica x_j foi positiva, então w_j foi muito alto e deve ser reduzido.

Escala das características. O gradiente descendente é sensível à escala das características. Se uma variável varia entre 0 e 10.000 e outra entre 0 e 1, os pesos correspondentes podem ser atualizados em ritmos muito diferentes, tornando a convergência lenta ou instável. Por isso, é comum padronizar ou normalizar as características antes do treinamento — por exemplo, subtraindo a média e dividindo pelo desvio padrão.

Analogia com Q-learning. Comparando a regra de atualização acima com a do Q-learning aproximado:

$$w_j \leftarrow w_j + \alpha \cdot \underbrace{\delta}_{\text{erro TD}} \cdot f_j(s, a)$$

a estrutura é idêntica: ajuste proporcional ao erro (TD ou de regressão) vezes o valor da característica. No Q-learning, $\delta = [r + \gamma \max_{a'} Q(s', a')] - Q(s, a)$; na regressão linear, $\delta = y^{(i)} - \hat{y}^{(i)}$. Na regressão o alvo é fornecido pelos dados. No Q-learning, a atualização trata o alvo TD como constante e é um semigradiente, conforme discutido na Seção 4.2.

Variantes de gradiente descendente. Na prática, calcular o gradiente sobre *todos* os n exemplos a cada passo pode ser custoso. Há três variantes:

- *Gradiente descendente em lote (batch gradient descent)*: usa todos os n exemplos. Atualização precisa, mas custosa para n grande.
- *Gradiente descendente estocástico (stochastic gradient descent, SGD)*: usa um único exemplo por passo, escolhido aleatoriamente. Atualização ruidosa, mas muito rápida. Convergência instável, mas frequentemente suficiente.
- *Gradiente descendente em mini-lote (mini-batch gradient descent)*: usa um subconjunto de b exemplos por passo (b típico: 32–256). Compromisso prático. É o padrão em aprendizado profundo.

A Figura 4 ilustra a dinâmica do gradiente descendente em lote no espaço dos parâmetros (w_1, w_2). As elipses representam curvas de nível da função de custo $J(\mathbf{w})$: pontos sobre a mesma curva têm o mesmo valor de erro. O ponto $\mathbf{w}^{(0)}$ indica a inicialização dos parâmetros, e as setas mostram as atualizações sucessivas feitas pelo algoritmo. Em cada passo, o gradiente descendente usa todos os exemplos de treinamento para calcular a direção média de maior crescimento da função de custo e, em seguida, move os parâmetros na direção oposta, isto é, na direção de maior descida. À medida que as atualizações avançam, a trajetória se aproxima do mínimo $\mathbf{w}^*$, onde o custo é menor e novas atualizações passam a ter magnitude cada vez menor.

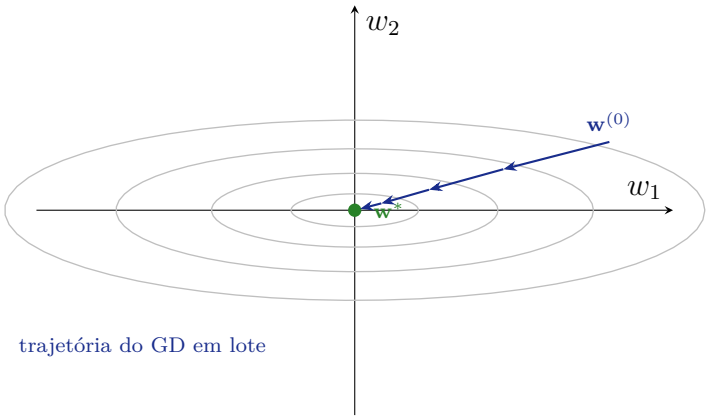


Figura 4: Trajetória do gradiente descendente em lote num espaço bidimensional de parâmetros. As curvas de nível representam o custo $J(\mathbf{w})$; a trajetória parte de $\mathbf{w}^{(0)}$ e aproxima-se do mínimo com uma taxa de aprendizado adequada $\mathbf{w}^*$, movendo-se sempre na direção de maior descida.

A.2.1 Exemplo resolvido: um passo de gradiente descendente

Continuando o exemplo anterior com $d = 1$: $h(x) = b + w_1 x$. Suponha que os pesos atuais sejam $b = 0$, $w_1 = 4$, e $\alpha = 0{,}0001$.

Passo 1: calcular os erros.

$$e_1 = h(50) - 200 = 200 - 200 = 0$$
$$e_2 = h(100) - 350 = 400 - 350 = +50$$
$$e_3 = h(150) - 500 = 600 - 500 = +100$$

Passo 2: calcular os gradientes.

$$\frac{\partial J}{\partial w_1} = \frac{2}{3}(0 \cdot 50 + 50 \cdot 100 + 100 \cdot 150) = \frac{2}{3}(0 + 5000 + 15000) = \frac{40000}{3} \approx 13333$$

$$\frac{\partial J}{\partial b} = \frac{2}{3}(0 + 50 + 100) = 100$$

Passo 3: atualizar os pesos.

$$w_1 \leftarrow 4 - 0{,}0001 \times 13333 \approx 4 - 1{,}33 = 2{,}67$$

$$b \leftarrow 0 - 0{,}0001 \times 100 = -0{,}01$$

Interpretação: w_1 desceu de 4 para 2,67 (em direção ao valor ótimo 3), pois o modelo estava superestimando os preços de imóveis maiores. O processo se repete até convergência.

A.3 Engenharia de características

Modelos lineares têm limitações óbvias: eles só conseguem representar fronteiras de decisão lineares (hiperplanos). No entanto, a maioria dos problemas reais envolve relações não lineares. Como lidar com isso mantendo a tratabilidade dos modelos lineares?

A resposta é a *engenharia de características*: transformar as variáveis originais em um novo espaço de representação onde as relações sejam (mais próximas de) lineares. Algumas transformações comuns:

- *Características polinomiais*: adicionar x_1^2, $x_1 x_2$, x_2^2, etc. Permite capturar relações polinomiais.
- *Logaritmo*: $\log x_j$ para variáveis com distribuições muito assimétricas (e.g., rendas, preços).
- *Interações*: $x_j \cdot x_k$ captura relações de multiplicação entre características.
- *Binarização*: converter variáveis categóricas em variáveis binárias (*one-hot encoding*).

Engenharia de características e Q-learning. Os leitores das notas de AR já encontraram engenharia de características no Q-learning com aproximação linear: as funções $f_i(s, a)$ são exatamente características projetadas manualmente para capturar aspectos relevantes do par estado-ação. O princípio é o mesmo: um modelo linear no espaço de características pode representar relações não lineares no espaço original.

O risco de características polinomiais: overfitting. Adicionar características polinomiais de grau alto permite que o modelo se ajuste perfeitamente aos dados de treinamento, interpolando por todos os pontos. Mas isso geralmente prejudica a generalização, pois o modelo "memoriza" os dados em vez de aprender a tendência subjacente. Esse fenômeno é chamado de *overfitting* e é estudado em detalhes na Seção 3.

Pergunta 8

Explique por que a regressão linear pura não consegue classificar pontos que não são linearmente separáveis. Descreva uma transformação de características que permitiria a um modelo linear separar os pontos $\{(1,1),(-1,-1)\}$ (classe $+$) dos pontos $\{(1,-1),(-1,1)\}$ (classe $-$). (*Dica*: observe que esses pontos são separáveis por uma fronteira do tipo $x_1 \cdot x_2 = 0$.)

Pergunta 9

Um modelo de regressão linear treinado para prever o consumo de energia elétrica de edifícios usa como características a área útil e o número de andares. Após treinamento, o modelo obtém MSE $= 250$ no conjunto de treino e MSE $= 1800$ no conjunto de teste. Identifique o problema e proponha três ações concretas (relacionadas aos dados, ao modelo ou ao processo de avaliação) para melhorar o desempenho de generalização.

B Aprofundamento em Generalização, Regularização e Métricas

B.1 Underfitting e overfitting

Toda a dificuldade do AM supervisionado pode ser resumida em um único dilema: um modelo muito simples não captura os padrões dos dados (*underfitting*); um modelo muito complexo captura até o ruído dos dados de treinamento e não generaliza (*overfitting*).

A Figura 5 compara três comportamentos típicos de modelos de regressão diante do mesmo conjunto de dados. No painel à esquerda, a reta horizontal representa um modelo simples demais: ele ignora a tendência crescente dos pontos e comete erros sistemáticos, caracterizando *underfitting* ou alto viés. No painel central, a reta inclinada captura a estrutura principal dos dados sem tentar explicar cada pequena flutuação individual; esse é o caso desejável, no qual há um equilíbrio entre simplicidade e capacidade de ajuste. No painel à direita, a curva passa exatamente pelos exemplos observados, mas faz isso seguindo também as irregularidades locais dos dados. Esse comportamento caracteriza *overfitting*: o erro no conjunto de treino pode ser muito baixo, mas a previsão tende a ser instável em novos exemplos, pois o modelo aprendeu ruído junto com o padrão.

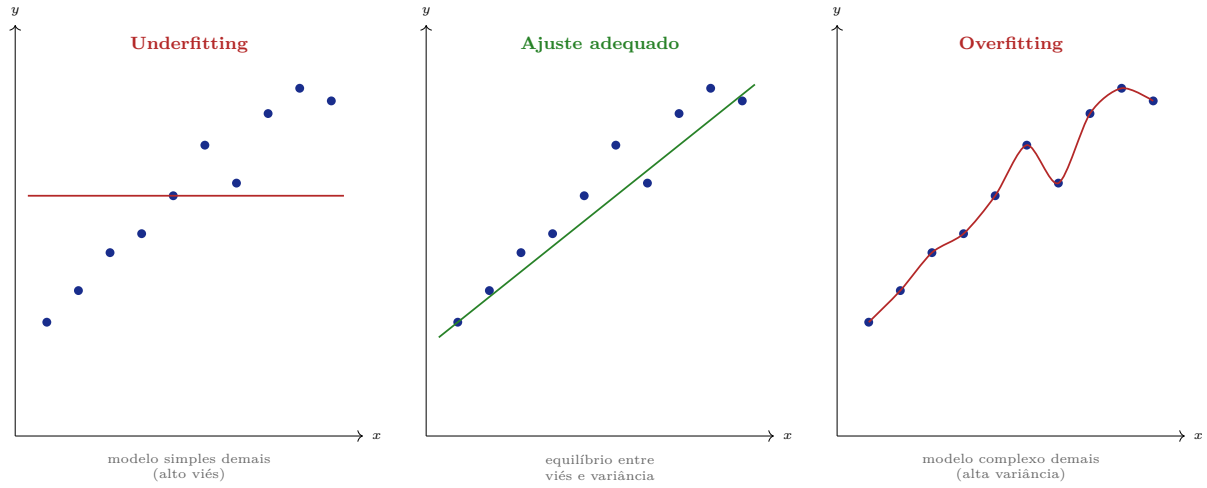


Figura 5: Três casos de ajuste em regressão. À esquerda, o modelo é simples demais (linha plana) e não captura a tendência crescente dos dados: underfitting. No centro, o modelo é linear e captura a tendência sem se prender ao ruído: ajuste adequado. À direita, o modelo passa exatamente por todos os pontos mas oscilará muito em dados novos: overfitting.

A Figura 6 mostra como o erro varia à medida que aumentamos a complexidade do modelo. Modelos muito simples, à esquerda, não conseguem capturar os padrões relevantes dos dados: tanto o erro de treino quanto o erro de validação permanecem altos, caracterizando *underfitting*. À medida que a complexidade aumenta, o modelo passa a representar melhor a relação entre entradas e saídas, e o erro de validação diminui. A região central representa o melhor compromisso: o modelo é expressivo o bastante para aprender padrões úteis, mas ainda não está ajustado demais às particularidades do conjunto de treino. À direita, modelos muito complexos continuam reduzindo o erro de treino, pois conseguem se adaptar cada vez mais aos exemplos observados. No entanto, o erro de validação volta a crescer, indicando *overfitting*: o modelo deixa de aprender

apenas padrões gerais e passa a capturar ruídos ou particularidades do treino. Por isso, a escolha da complexidade deve ser guiada pelo erro de validação, não pelo erro de treino.

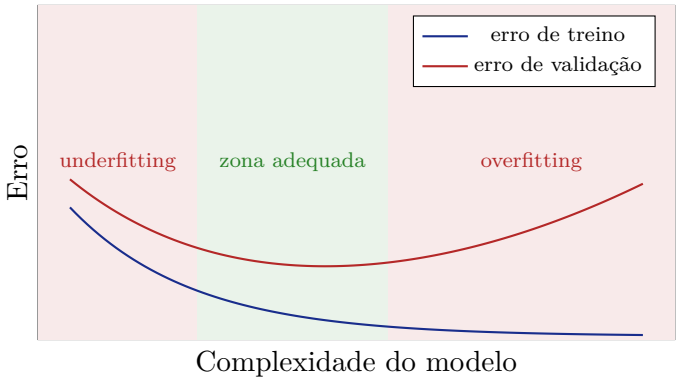


Figura 6: Curvas de erro em função da complexidade do modelo. O erro de treino decresce monotonicamente e se aproxima de zero (o modelo memoriza os dados). O erro de validação tem formato de U: decresce até um ponto ótimo, depois sobe (overfitting). A complexidade ótima é aquela que minimiza o erro de validação.

B.2 O dilema viés-variância

O dilema viés-variância é a formalização matemática do problema de underfitting e overfitting. No caso de regressão com perda quadrática, para um ponto de teste fixo $\mathbf{x}$, supondo $y = f(\mathbf{x}) + \varepsilon$, com ruído de média zero e variância finita, o erro esperado admite a decomposição:

$$\mathbb{E}\left[(h(\mathbf{x}) - y)^2\right] = \underbrace{\text{Viés}^2(h(\mathbf{x}))}_{\text{erro sistemático}} + \underbrace{\text{Variância}(h(\mathbf{x}))}_{\text{sensibilidade ao dado}} + \underbrace{\sigma^2_{\text{ruído}}}_{\text{irredutível}}$$

onde a expectativa é tomada sobre diferentes conjuntos de treinamento amostrados da mesma distribuição.

- *Viés*: mede o quanto as previsões do modelo são sistematicamente erradas, independentemente dos dados de treinamento. Um modelo linear aplicado a um problema intrinsecamente quadrático tem alto viés.
- *Variância*: mede o quanto as previsões do modelo mudam quando o conjunto de treinamento muda. Um modelo polinomial de grau alto, treinado em amostras diferentes da mesma distribuição, produz previsões muito diferentes (alta variância).
- *Ruído irredutível*: a variabilidade inerente dos dados ($y = f(\mathbf{x}) + \varepsilon$), que nenhum modelo pode eliminar.

Viés-variância no AR. O dilema viés-variância já apareceu implicitamente nas notas de AR, na comparação entre Monte Carlo e TD. MC tem baixo viés (usa retornos reais) mas alta variância (o retorno depende de toda a sequência de recompensas ruidosas). TD tem viés introduzido pelo bootstrap (o alvo usa estimativas), mas menor variância (o alvo é mais estável). A escolha entre MC e TD é, portanto, uma escolha no espaço viés-variância.

B.3 Regularização

A regularização é a técnica principal para controlar overfitting sem aumentar o conjunto de dados. A ideia é adicionar um *termo de penalidade* à função de custo que favorece modelos mais simples (com pesos menores):

$$J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \cdot \Omega(\mathbf{w})$$

onde $\lambda > 0$ é o *hiperparâmetro de regularização* que controla o equilíbrio entre ajuste aos dados e simplicidade do modelo. As duas formas mais comuns são:

Regularização ℓ_2 (Ridge):

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_2^2 = \sum_j w_j^2$$

Penaliza pesos grandes, forçando-os a permanecer próximos de zero. A penalidade favorece pesos menores, sem induzir tipicamente zeros exatos. Não implica que cada peso será menor do que em qualquer solução sem regularização.

Regularização ℓ_1 (Lasso):

$$\Omega(\mathbf{w}) = \|\mathbf{w}\|_1 = \sum_j |w_j|$$

Produz soluções *esparsas*: muitos pesos tornam-se exatamente zero, efetivamente selecionando um subconjunto de características.

Uma forma intuitiva de visualizar a regularização é interpretar a restrição sobre os pesos como uma região permitida no plano (w_1, w_2). Sem regularização, o otimizador buscaria o centro das curvas de nível da perda, isto é, o ponto de menor erro nos dados de treinamento. Com regularização, porém, a solução deve permanecer dentro da região permitida. A solução regularizada ocorre no primeiro ponto em que uma curva de nível da perda toca essa região, como mostra a Figura 7. Na regularização ℓ_2, a região permitida é circular: sua borda é suave, sem cantos alinhados aos eixos. Por isso, o ponto de contato costuma ter coordenadas pequenas, mas raramente exatamente nulas. Já na regularização ℓ_1, a região permitida tem forma de losango, com vértices sobre os eixos $w_1 = 0$ e $w_2 = 0$. Como esses vértices são pontos salientes da região, é frequente que uma curva de nível toque o losango justamente em um deles. Quando isso acontece, uma das coordenadas do vetor de pesos fica exatamente igual a zero. Essa é a origem geométrica da esparsidade induzida pela regularização ℓ_1.

B.3.1 Exemplo resolvido

Suponha um modelo linear com dois pesos $w_1 = 3{,}0$ e $w_2 = 0{,}1$, e custo de treinamento $J = 1{,}5$, com $\lambda = 0{,}5$.

Custo regularizado ℓ_2:

$$J_{\ell_2} = 1{,}5 + 0{,}5 \times (3{,}0^2 + 0{,}1^2) = 1{,}5 + 0{,}5 \times 9{,}01 = 1{,}5 + 4{,}505 = 6{,}005$$

Custo regularizado ℓ_1:

$$J_{\ell_1} = 1{,}5 + 0{,}5 \times (|3{,}0| + |0{,}1|) = 1{,}5 + 0{,}5 \times 3{,}1 = 1{,}5 + 1{,}55 = 3{,}05$$

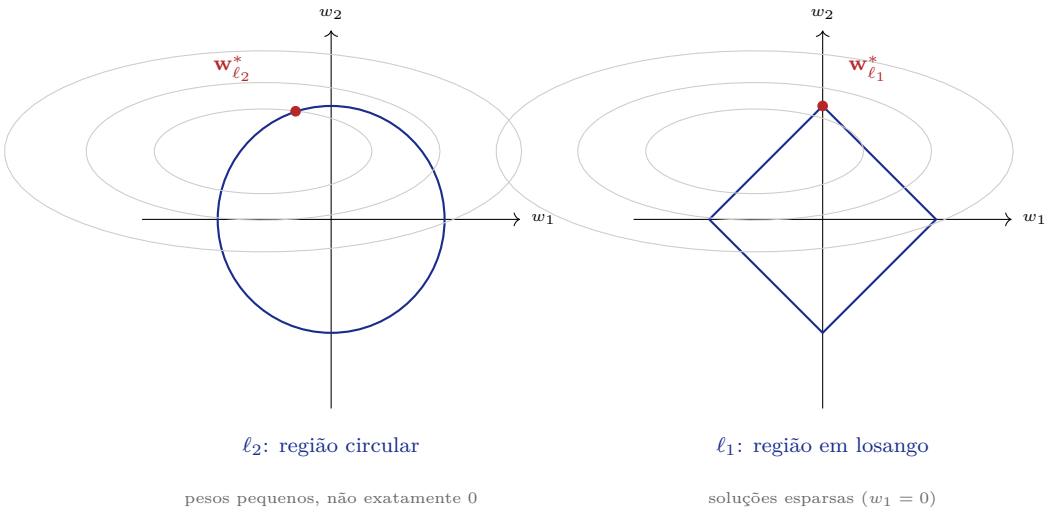


Figura 7: Intuição geométrica da regularização ℓ_2 e ℓ_1. As curvas cinzas representam níveis de mesma perda, e a região azul representa os pesos permitidos pela regularização. A solução regularizada é o ponto em que a menor curva de nível possível toca essa região. Na ℓ_2, a borda circular tende a produzir pesos menores, mas não exatamente nulos. Na ℓ_1, o losango possui vértices sobre os eixos; quando a solução toca um desses vértices, uma coordenada do vetor de pesos torna-se exatamente zero, produzindo uma solução esparsa.

Interpretação: a regularização penaliza $w_1 = 3{,}0$ muito mais do que $w_2 = 0{,}1$. O otimizador é incentivado a reduzir w_1, a menos que mantê-lo grande seja fortemente justificado pelos dados. Isso controla a magnitude dos pesos; o efeito sobre a generalização deve ser verificado na validação.

B.4 Validação cruzada

Nos esquemas anteriores, vimos duas formas comuns de separar dados: *train/test* e *train/-val/test*. No segundo caso, o conjunto de validação é usado durante o desenvolvimento do modelo: escolhemos hiperparâmetros, comparamos arquiteturas e decidimos quando parar o treinamento com base no desempenho nesse subconjunto. O conjunto de teste, por sua vez, permanece intocado até a avaliação final.

Essa estratégia funciona bem quando há dados suficientes. Mas, em conjuntos pequenos, reservar uma parte fixa dos dados apenas para validação pode ser caro: o modelo passa a ser treinado com menos exemplos, e a estimativa de desempenho pode depender muito de quais exemplos caíram no subconjunto de validação. A validação cruzada surge exatamente para reduzir esse problema.

A ideia é substituir o conjunto de validação fixo por várias validações sucessivas. Em vez de separar uma única fatia dos dados para validação, o conjunto de treino é dividido em partes, e cada parte será usada como validação uma vez. Assim, todos os exemplos participam do treinamento em algumas iterações e da validação em outra. O resultado é uma estimativa de desempenho menos dependente de uma única divisão dos dados.

Quando o conjunto de dados é pequeno, reservar 15–20% para validação pode desperdiçar dados preciosos de treinamento. A *validação cruzada* k-fold resolve esse problema:

1. Divida o conjunto de treino em k partes (*folds*) de tamanho igual.

2. Para $i = 1, \ldots, k$: treine o modelo em todos os folds exceto o i-ésimo; avalie no fold i.

3. O erro de validação estimado é a média dos k erros obtidos.

A Figura 8 ilustra o funcionamento da validação cruzada 5-fold. O conjunto de treino é dividido em cinco partes, e o processo é repetido cinco vezes. Em cada iteração, uma parte diferente é usada como validação, enquanto as quatro restantes são usadas para treinamento. Ao final, obtêm-se cinco estimativas de erro, uma para cada escolha do fold de validação, e o desempenho final é calculado pela média desses erros. A figura também reforça que essa alternância ocorre apenas dentro dos dados disponíveis para treinamento e validação; o conjunto de teste permanece separado e não participa desse processo.



Figura 8: Validação cruzada 5-fold. A cada iteração, um fold diferente é usado para validação e os restantes para treinamento. O erro estimado é a média dos 5 erros de validação. Ao longo das 5 iterações, cada exemplo é usado para validação uma vez e para treinamento nas demais iterações. O conjunto de teste permanece completamente separado e não participa da validação cruzada.

Validação cruzada não substitui o conjunto de teste. A validação cruzada substitui a separação treino/validação durante a escolha de modelos e hiperparâmetros, permitindo usar os dados de forma mais eficiente. No entanto, ela não substitui o conjunto de teste. O teste continua reservado para a avaliação final, depois que todas as escolhas de modelo e hiperparâmetros foram feitas. Aplicar validação cruzada ao conjunto de teste é um erro equivalente ao vazamento de dados.

B.5 Métricas de avaliação

A escolha da métrica de avaliação depende da tarefa e dos requisitos do domínio.

Regressão.

- *MSE* (Erro Quadrático Médio): $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$. Penaliza erros grandes.
- *RMSE*: $\sqrt{\text{MSE}}$. Na mesma unidade da variável-alvo.
- *MAE* (Erro Absoluto Médio): $\frac{1}{n}\sum|y_i - \hat{y}_i|$. Mais robusto a outliers.
- R^2 (coeficiente de determinação): fração da variância explicada pelo modelo. $R^2 = 1$ indica ajuste perfeito; $R^2 = 0$, igual à referência da média do conjunto avaliado; valores negativos também são possíveis.

Classificação. A métrica mais comum é a *acurácia*: fração de exemplos classificados corretamente. Mas ela pode ser enganosa em dados desbalanceados. Se 95% dos exemplos são da classe 0, um modelo que sempre prevê 0 tem acurácia 95% mas não detecta nenhum exemplo da classe minoritária.

Para analisar melhor o desempenho, usamos a *matriz de confusão* (Tabela 1) e métricas derivadas. A distinção entre falso positivo e falso negativo é essencial porque os dois tipos de erro podem ter custos muito diferentes dependendo do domínio.

Tabela 1: Matriz de confusão para classificação binária.

		Previsto	
		Positivo	Negativo
Real	Positivo	VP (Verdadeiro Positivo)	FN (Falso Negativo)
	Negativo	FP (Falso Positivo)	VN (Verdadeiro Negativo)

- **Precisão**: $\frac{\text{VP}}{\text{VP+FP}}$ dos que foram classificados como positivos, quantos realmente são?
- **Revocação** (*recall*): $\frac{\text{VP}}{\text{VP+FN}}$ dos que são positivos, quantos o modelo encontrou?
- **F1-score**: média harmônica de precisão e revocação: $\frac{2 \cdot \text{Precisão} \cdot \text{Revocação}}{\text{Precisão+Revocação}}$.

Precisão *versus* revocação: o dilema do médico. Em diagnóstico médico, um falso negativo (não detectar uma doença grave) é geralmente muito pior do que um falso positivo (suspeitar de uma doença que não existe). Nesse contexto, maximizar a revocação é prioritário, mesmo que isso reduza a precisão. Já em filtragem de spam, um falso positivo (deletar um e-mail legítimo) pode ser pior do que um falso negativo (deixar passar um spam). A escolha da métrica deve refletir as consequências reais dos erros no domínio.

Pergunta 10

Um modelo de AM para detecção de fraude em transações bancárias tem 99,8% de acurácia, mas a equipe de segurança está insatisfeita. Explique por que a acurácia pode ser uma métrica enganosa neste contexto. Qual combinação de métricas seria mais informativa?

Pergunta 11

Um engenheiro de ML treina três versões de um modelo de regressão usando características polinomiais de graus 1, 5 e 15. O modelo de grau 1 obtém MSE de treino = 120 e MSE de validação = 125. O de grau 5 obtém MSE de treino = 45 e MSE de validação = 52. O de grau 15 obtém MSE de treino = 2 e MSE de validação = 380. **(a)** Qual modelo apresenta underfitting? Qual apresenta overfitting? **(b)** Qual você selecionaria para implantação e por quê? **(c)** Se você quisesse melhorar o modelo de grau 15 sem reduzir o grau, que estratégia usaria?

C Árvores de Decisão

Modelos lineares representam fronteiras de decisão como hiperplanos. As *árvores de decisão* adotam outra estratégia: em vez de calcular uma única pontuação a partir de todas as variáveis, elas conduzem o exemplo $\mathbf{x}$ por uma sequência de perguntas simples. Cada pergunta testa uma característica x_j do exemplo. Cada resposta leva a uma nova pergunta ou a uma decisão final.

Antes de examinar uma árvore formalmente, considere um problema simples: classificar um e-mail como *spam* ou *legítimo*. Um classificador linear tentaria combinar várias características em uma única pontuação. Uma árvore de decisão segue outra lógica: ela faz uma pergunta por vez. Por exemplo, suponha que usamos três características de um e-mail:

- quantas vezes aparece a palavra "grátis";
- quantos links externos o e-mail contém;
- se o assunto está escrito em letras maiúsculas.

Uma possível regra de classificação seria a seguinte. Primeiro, verificamos se a palavra "grátis" aparece pelo menos três vezes. Se sim, olhamos a quantidade de links externos. Se houver muitos links, classificamos como spam; caso contrário, como legítimo. Se a palavra "grátis" não aparece com tanta frequência, verificamos se o assunto está em maiúsculas. Dependendo dessa resposta, chegamos novamente a uma das duas previsões possíveis.

Essa sequência de perguntas pode ser representada como uma árvore. Cada nó interno corresponde a um teste sobre uma característica. Cada aresta corresponde a uma resposta possível ao teste. Cada folha contém a decisão final.

A Figura 9 apresenta uma árvore de decisão para classificar e-mails como spam ou legítimos. A classificação é feita por uma sequência de perguntas simples sobre atributos do e-mail. Primeiro, verifica-se se a palavra "grátis" aparece pelo menos três vezes. Dependendo da resposta, o exemplo segue por um dos ramos da árvore e passa por novos testes, como a quantidade de links externos ou o uso de assunto em letras maiúsculas. O caminho termina em uma folha, que contém a previsão final. Essa estrutura torna o modelo interpretável: para justificar uma decisão, basta seguir o caminho percorrido da raiz até a folha correspondente. No exemplo, um e-mail pode ser marcado como spam não por uma pontuação abstrata, mas porque satisfez uma combinação explícita de condições.

De forma mais geral, uma árvore de decisão divide o espaço de entradas em regiões sucessivamente menores. Cada nó interno aplica um teste, geralmente da forma $x_j \leq \tau$, em que x_j representa a j-ésima característica do exemplo e τ é um limiar escolhido pelo algoritmo. Cada resposta ao teste define um ramo da árvore. Ao final do caminho, chega-se a uma folha, isto é, um nó terminal que contém a previsão do modelo. Como todos os exemplos que chegam à mesma folha recebem a mesma saída, dizemos que a árvore faz uma previsão constante em cada região associada a uma folha.

A principal vantagem das árvores de decisão é a *interpretabilidade*: cada decisão pode ser rastreada por um caminho explícito na árvore. A principal desvantagem é que, sem controle de profundidade, árvores profundas tendem a sofrer overfitting severo.

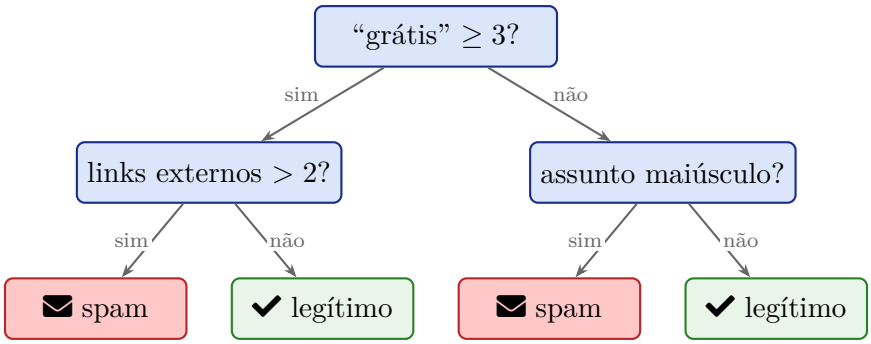


Figura 9: Exemplo de árvore de decisão para classificação de spam. Cada nó interno testa uma característica; cada folha produz uma previsão. A interpretabilidade é uma vantagem central: é fácil explicar por que um e-mail foi classificado como spam.

C.1 Construção de árvores de decisão para classificação

Nesta seção, consideramos árvores de decisão para *classificação*. Nesse cenário, cada exemplo pertence a uma classe, como *spam* ou *legítimo*. Ao construir a árvore, queremos escolher testes que separem os exemplos em subconjuntos cada vez mais homogêneos em relação à classe.

C.1.1 Entropia e ganho de informação

O problema central na construção de uma árvore é: *qual atributo usar em cada nó de divisão?* Em um nó, temos um conjunto de exemplos de treinamento. Se esses exemplos misturam várias classes, o nó é impuro. Se quase todos pertencem à mesma classe, o nó é mais puro. O critério de *ganho de informação* mede quanto uma divisão reduz essa impureza.

A construção da árvore é um procedimento *guloso* (*greedy*): em cada nó, escolhe-se a melhor divisão local segundo o critério de ganho de informação. Isso não garante a árvore globalmente ótima, mas produz um algoritmo eficiente e interpretável.

Para um conjunto de exemplos S, seja p_k a proporção de exemplos de S pertencentes à classe k. A entropia de S é definida por:

$$H(S) = -\sum_{k} p_k \log_2 p_k$$

A entropia mede a impureza (ou desordem) do conjunto. Ela é máxima quando as classes estão igualmente distribuídas ($H = \log_2 K$ para K classes) e mínima ($H = 0$) quando todos os exemplos pertencem à mesma classe.

A Figura 10 ilustra o comportamento da entropia no caso binário ($k = 2$). Se denotamos por p a proporção de exemplos de uma das classes em um nó, então a outra classe tem proporção $1 - p$. Observa-se que a entropia é igual a zero quando $p = 0$ ou $p = 1$, isto é, quando todos os exemplos pertencem à mesma classe e o nó é puro. À medida que as classes se tornam mais equilibradas, a entropia aumenta, atingindo seu valor máximo em $p = 0.5$. Isso confirma a intuição de que a entropia mede impureza: ela é baixa em nós homogêneos e alta em nós com forte mistura de classes.

A entropia, sozinha, apenas descreve o grau de mistura das classes em um nó. Para construir a árvore, precisamos dar um passo a mais: avaliar se uma possível pergunta

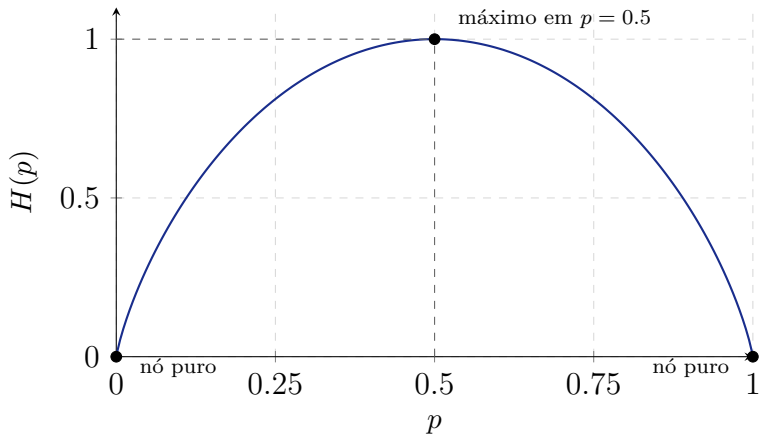


Figura 10: Entropia no caso binário, em que uma classe tem proporção p e a outra tem proporção $1 - p$. A entropia é mínima quando o nó é puro ($p = 0$ ou $p = 1$) e máxima quando as duas classes estão igualmente representadas ($p = 0.5$).

reduz essa mistura. Uma boa pergunta é aquela que pega um conjunto inicialmente impuro e o separa em subconjuntos mais puros.

Por exemplo, no problema de spam, a pergunta "a palavra grátis aparece pelo menos três vezes?" divide os e-mails em dois subconjuntos: os que respondem "sim" e os que respondem "não". Se, depois dessa divisão, cada subconjunto ficar mais homogêneo em termos de classe, então essa pergunta foi útil. O *ganho de informação* mede exatamente essa utilidade: quanto a entropia diminui depois da divisão.

O *ganho de informação* de um atributo A com valores possíveis $\{v_1, v_2, \ldots\}$ é:

$$\mathrm{IG}(S, A) = H(S) - \sum_v \frac{|S_v|}{|S|} H(S_v)$$

onde S_v é o subconjunto de S em que $A = v$. O atributo escolhido para divisão é aquele que maximiza $\mathrm{IG}(S, A)$.

C.1.2 Testes candidatos e construção recursiva da árvore

Até aqui, falamos em escolher um atributo para dividir um nó. Essa descrição é útil, mas ainda incompleta. Na prática, o algoritmo não escolhe apenas um atributo isolado; ele escolhe um *teste*. Um teste define uma pergunta que separa os exemplos de um nó em subconjuntos.

Quando o atributo é categórico, a divisão costuma ser direta. Por exemplo, o atributo "assunto em letras maiúsculas" já sugere uma pergunta binária: "o assunto está em letras maiúsculas?". As respostas possíveis são "sim" e "não".

Quando o atributo é numérico, o algoritmo também precisa escolher um limiar. Se x_j representa a j-ésima característica de um exemplo, um teste típico tem a forma

$$x_j \leq \tau,$$

em que τ é o limiar usado para separar os exemplos. No problema de spam, se x_j representa o número de vezes que a palavra "grátis" aparece no e-mail, um teste possível seria "grátis" ≥ 3. Outros testes também poderiam ser avaliados, como "grátis" ≥ 1, "grátis" ≥ 2 ou "grátis" ≥ 4.

O limiar não é escolhido arbitrariamente. Em uma implementação real, o algoritmo testa vários limiares candidatos para cada atributo numérico, calcula o ganho de informação produzido por cada teste e escolhe aquele que gera a maior redução de impureza no nó atual. Assim, em atributos numéricos, a escolha feita pelo algoritmo envolve o par

$$(\text{atributo}, \text{limiar}).$$

Depois que o melhor teste é escolhido, o nó é dividido em filhos. Cada filho recebe apenas os exemplos que satisfazem a resposta correspondente ao teste. O mesmo procedimento é então repetido em cada filho: calcula-se a impureza do novo conjunto, avaliam-se novos testes candidatos e escolhe-se a melhor divisão local.

Essa repetição caracteriza a construção *recursiva* da árvore. A raiz é dividida primeiro. Em seguida, cada nó filho pode ser dividido novamente. O processo continua até que algum critério de parada seja satisfeito. Há vários critérios que podem ser usados. Por exemplo: o nó já contém exemplos de uma única classe, não há ganho de informação suficiente, a profundidade máxima foi atingida ou há poucos exemplos no nó.

Portanto, uma árvore de decisão é construída por uma sequência de decisões locais. Em cada nó, o algoritmo pergunta: "qual teste torna os filhos mais puros?". A resposta define a divisão daquele nó. A aplicação recursiva dessa ideia produz a árvore completa.

C.1.3 Exemplo resolvido

Considere um conjunto de 10 e-mails: 6 spam (+) e 4 legítimos (−). Suponha que o algoritmo esteja comparando dois testes candidatos para dividir o nó raiz:

- Teste A: "grátis" ≥ 3, que divide os exemplos em $S_{\text{sim}} = \{5+, 1-\}$ e $S_{\text{não}} = \{1+, 3-\}$.
- Teste B: links > 2, que divide os exemplos em $S_{\text{sim}} = \{3+, 3-\}$ e $S_{\text{não}} = \{3+, 1-\}$.

Passo 1: calcular a entropia do conjunto raiz.

$$\begin{aligned} H(S) &= -\frac{6}{10}\log_2\frac{6}{10} - \frac{4}{10}\log_2\frac{4}{10} \\ &= -0{,}6 \times (-0{,}737) - 0{,}4 \times (-1{,}322) \\ &= 0{,}442 + 0{,}529 = 0{,}971 \end{aligned}$$

Passo 2: calcular IG do Atributo A.

$$H(S_{\text{sim}}) = -\frac{5}{6}\log_2\frac{5}{6} - \frac{1}{6}\log_2\frac{1}{6} \approx 0{,}650$$

$$H(S_{\text{não}}) = -\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4} \approx 0{,}811$$

$$\begin{aligned} \text{IG}(S, A) &= 0{,}971 - \left(\frac{6}{10} \times 0{,}650 + \frac{4}{10} \times 0{,}811\right) \\ &= 0{,}971 - (0{,}390 + 0{,}324) = 0{,}971 - 0{,}714 = 0{,}257 \end{aligned}$$

Passo 3: calcular IG do Atributo B.

$$H(S_{\text{sim}}) = -\frac{3}{6}\log_2\frac{3}{6} - \frac{3}{6}\log_2\frac{3}{6} = 1{,}0$$

$$H(S_{\text{não}}) = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} \approx 0{,}811$$

$$\text{IG}(S, B) = 0{,}971 - \left(\frac{6}{10} \times 1{,}0 + \frac{4}{10} \times 0{,}811\right)$$

$$= 0{,}971 - (0{,}600 + 0{,}324) = 0{,}971 - 0{,}924 = 0{,}047$$

Decisão: o Atributo A (IG = 0,257) é muito melhor que o B (IG = 0,047). O algoritmo escolhe A para dividir o nó raiz. Intuitivamente, A cria subconjuntos muito mais puros (mais homogêneos em termos de classe).

Neste exemplo, estamos comparando testes completos, isto é, pares formados por um atributo e um limiar. Em uma árvore real, outros limiares para os mesmos atributos também poderiam ser avaliados.

C.2 Controle de complexidade e poda

O procedimento de construção de uma árvore é recursivo: em cada nó, o algoritmo procura o teste que produz a maior redução de impureza e divide os exemplos em subconjuntos. Se esse processo continuar sem restrições, a árvore pode crescer até representar detalhes muito específicos do conjunto de treinamento.

No limite, uma árvore pode criar folhas quase puras contendo poucos exemplos, ou até mesmo um único exemplo. Isso parece bom do ponto de vista do erro de treinamento, pois a árvore passa a memorizar muitos casos particulares. Porém, essa memorização geralmente não se transfere para novos dados. A árvore se torna muito profunda, muito sensível a pequenas variações nos dados e tende a sofrer *overfitting*.

Por isso, árvores de decisão precisam de mecanismos de controle de complexidade. Esses mecanismos cumprem papel semelhante ao da regularização em outros modelos: eles impedem que o modelo se ajuste demais ao conjunto de treinamento. Em árvores, esse controle é feito principalmente por critérios de parada e por poda.

Há duas estratégias principais.

- *Pré-poda* (*pre-pruning*): interrompe o crescimento da árvore durante a construção. Um nó deixa de ser dividido quando a divisão não parece suficientemente útil ou quando há poucos dados para justificar uma nova ramificação. Exemplos de critérios são: ganho de informação menor que um limiar, profundidade máxima atingida ou número de exemplos no nó menor que $m_{\min}$.

- *Pós-poda* (*post-pruning*): primeiro constrói uma árvore grande, possivelmente ajustada demais ao treino, e depois remove ramos que não melhoram o desempenho em dados de validação. A ideia é perguntar: "se substituirmos este ramo inteiro por uma folha, o desempenho fora do treino piora?". Se não piorar, o ramo pode ser podado.

A diferença entre as duas estratégias está no momento da decisão. Na pré-poda, o algoritmo evita criar certos ramos. Na pós-poda, ele cria os ramos primeiro e decide

depois quais devem ser removidos. A pré-poda é mais simples e eficiente, mas pode parar cedo demais. A pós-poda costuma ser mais robusta, pois avalia a utilidade dos ramos depois que a árvore completa revelou sua estrutura.

A instabilidade das árvores de decisão. Árvores de decisão são modelos de alta variância. Pequenas mudanças no conjunto de treinamento podem alterar os testes escolhidos perto da raiz, e essas mudanças se propagam para toda a estrutura da árvore. Assim, remover ou adicionar poucos exemplos pode produzir árvores bastante diferentes.

Essa instabilidade é uma desvantagem de uma árvore isolada, mas também explica por que árvores são boas candidatas para métodos ensemble. Se uma única árvore é instável, podemos treinar muitas árvores diferentes e combinar suas previsões. A combinação tende a suavizar os erros individuais e produzir um modelo final mais estável e preciso.

D Métodos Ensemble

Um *método ensemble* combina as previsões de múltiplos modelos (chamados de *modelos base* ou *aprendizes fracos*) para produzir uma previsão final mais robusta. A intuição é que modelos diferentes erram em lugares diferentes, e sua combinação cancela esses erros.

D.1 Bagging: modelos independentes em paralelo

O *bagging* (*bootstrap aggregating*) reduz a variância treinando T modelos independentes em subconjuntos diferentes dos dados (amostrados com reposição usando o método *bootstrap*) e combinando suas previsões por votação (classificação) ou média (regressão).

Random Forests. Random Forests (Breiman, 2001) são a realização mais bem-sucedida do bagging com árvores de decisão. A ideia adicional é introduzir aleatoriedade também na seleção de atributos: em cada nó, em vez de considerar todos os d atributos, considera-se apenas um subconjunto aleatório de $\sqrt{d}$ (para classificação) ou $d/3$ (para regressão) atributos candidatos.

Esse mecanismo de dupla aleatorização (amostras e atributos) produz árvores mais diversas e menos correlacionadas entre si, o que resulta em cancelamento mais eficaz dos erros individuais.

A Figura 11 mostra a ideia central por trás de *bagging* e Random Forests. A partir do conjunto original de dados $\mathcal{D}$, são geradas várias amostras *bootstrap*, isto é, subconjuntos obtidos por sorteio com reposição. Cada amostra é usada para treinar uma árvore diferente, produzindo modelos que veem versões ligeiramente distintas dos dados. Como árvores de decisão tendem a ter alta variância, pequenas mudanças no conjunto de treino podem gerar árvores bastante diferentes. O ensemble aproveita essa instabilidade: em vez de confiar em uma única árvore, combina as previsões de várias delas por votação, em problemas de classificação, ou por média, em problemas de regressão. A Random Forest acrescenta ainda uma segunda fonte de diversidade, selecionando aleatoriamente um subconjunto de atributos em cada nó da árvore. Essa combinação reduz a variância do modelo final e torna a previsão mais robusta.

D.2 Boosting: modelos sequenciais que corrigem erros

O *boosting* adota uma estratégia diferente: em vez de treinar modelos independentes, treina-os *sequencialmente*, onde cada modelo foca nos exemplos que os anteriores erraram. O algoritmo mais clássico é o *AdaBoost*: exemplos classificados incorretamente recebem peso maior na rodada seguinte, forçando o próximo modelo a se concentrar neles.

O *Gradient Boosting* (e sua implementação eficiente, *XGBoost* (Chen e Guestrin, 2016)) generaliza essa ideia: cada novo modelo aproxima o gradiente negativo da perda do ensemble acumulado. Na perda quadrática, isso corresponde a ajustar os resíduos. O resultado é uma sequência de modelos simples que, somados, produzem um modelo muito poderoso.

A Figura 12 destaca a diferença central entre boosting e bagging. No bagging, os modelos base são treinados em paralelo e depois combinados. No boosting, os modelos são treinados em sequência. A figura mostra apenas três modelos para simplificar a visualização, mas não há nada especial nesse número: em aplicações reais, o ensemble pode conter dezenas, centenas ou até milhares de modelos base.

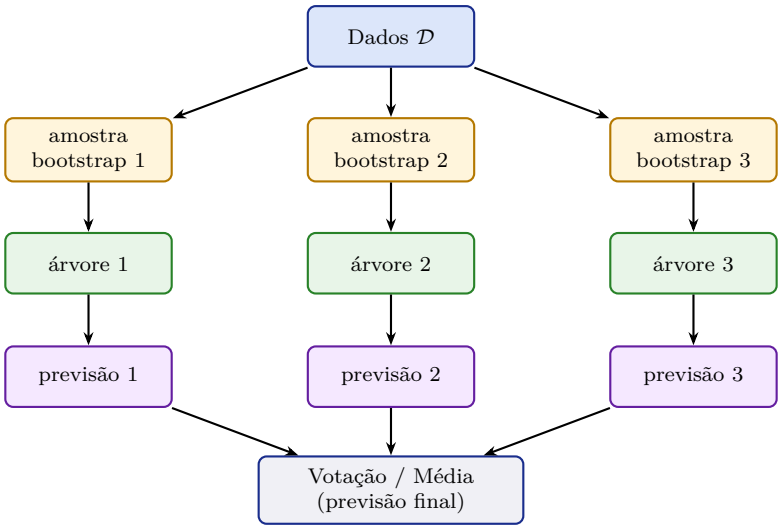


Figura 11: Arquitetura do bagging / Random Forest. Múltiplas árvores são treinadas em amostras bootstrap independentes; suas previsões são combinadas por votação ou média, reduzindo a variância em relação a uma única árvore.

O primeiro modelo produz uma aproximação inicial. Em seguida, o algoritmo identifica os exemplos mal previstos, ou os resíduos deixados pelo ensemble atual, e treina um novo modelo para reduzir esses erros. Esse processo pode continuar por várias etapas. A cada novo modelo, o ensemble acumulado é ajustado para corrigir parte dos erros restantes. A previsão final é obtida pela combinação acumulada de todos os modelos. Por isso, boosting costuma ser associado à redução de viés: ele constrói gradualmente um modelo mais expressivo a partir de vários modelos simples.

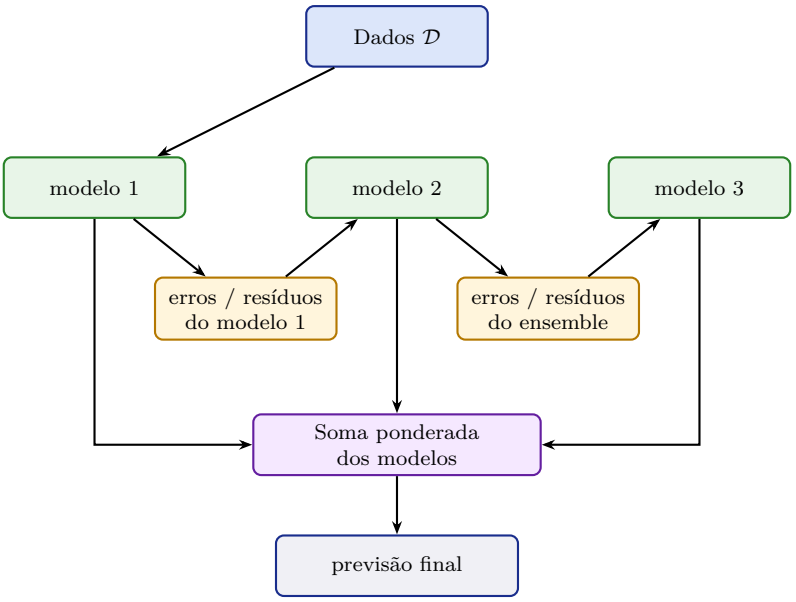
Boosting também pode sofrer overfitting. Embora o boosting seja frequentemente apresentado como uma técnica de redução de viés, modelos boosted também podem sofrer overfitting se forem muito profundos, numerosos ou pouco regularizados. Implementações modernas como XGBoost e LightGBM incorporam regularização e amostragem de dados por esse motivo. Sempre avalie o modelo boosted tanto no treino quanto na validação.

D.3 Stacking: combinação aprendida de modelos

O *stacking* (*stacked generalization*) combina modelos base de forma diferente. Em vez de usar uma regra fixa, como votação ou média, ele treina um novo modelo para aprender como combinar as previsões dos modelos base. Esse novo modelo é chamado de *meta-modelo*.

A ideia é usar modelos base diversos, por exemplo uma árvore de decisão, uma regressão logística e um k-NN. Cada um produz uma previsão para o mesmo exemplo. Essas previsões passam a formar uma nova representação dos dados. O meta-modelo recebe essa representação como entrada e aprende a produzir a previsão final.

Em geral, o meta-modelo não deve ser treinado usando previsões feitas sobre os mesmos exemplos usados para ajustar os modelos base, pois isso pode causar vazamento de informação e overfitting. Uma prática comum é gerar previsões fora da amostra, por



Boosting: cada novo modelo é treinado para reduzir os erros deixados pelos anteriores.

Figura 12: Arquitetura conceitual do boosting. Diferentemente do bagging, os modelos não são treinados de forma independente. Eles são ajustados em sequência, de modo que cada novo modelo tenta corrigir erros ou resíduos deixados pelo ensemble acumulado.

validação cruzada, para treinar o meta-modelo.

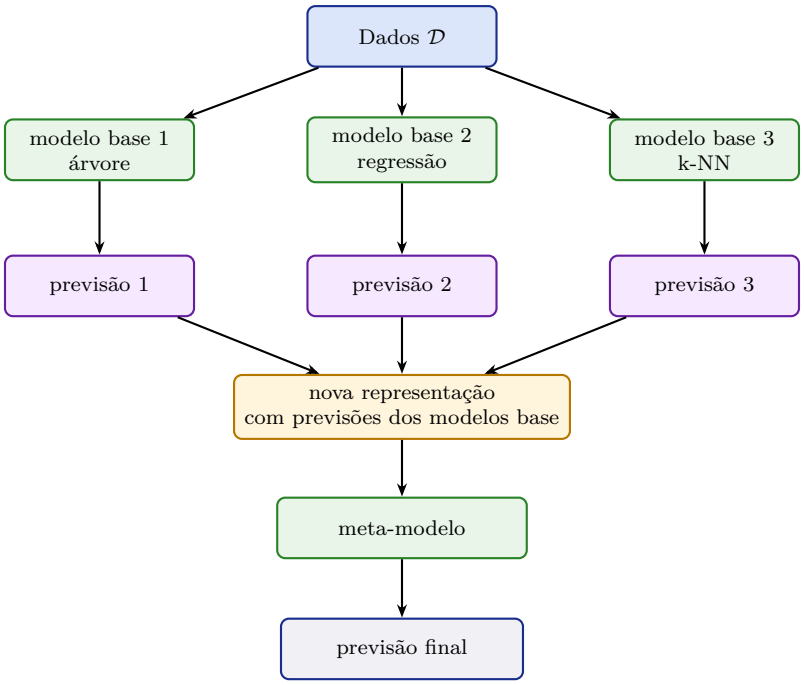
A Figura 13 mostra a lógica do stacking. A diferença em relação ao bagging e ao boosting está na forma de combinação. No bagging, a combinação é uma votação ou média. No boosting, a combinação é acumulada sequencialmente. No stacking, a combinação é aprendida por outro modelo. Esse meta-modelo observa as previsões dos modelos base e aprende, por exemplo, quando confiar mais em um classificador do que em outro. Por isso, stacking é especialmente útil quando os modelos base têm comportamentos complementares.

D.4 Comparativo

A Tabela 2 resume as principais diferenças entre os métodos de ensemble.

Pergunta 12

Árvores de decisão têm alta variância; modelos lineares simples têm alto viés. Como cada um dos métodos ensemble (bagging e boosting) ataca um desses dois problemas? Por que Random Forest funciona melhor que bagging de árvores simples?



Stacking: o modo de combinar os modelos base também é aprendido a partir dos dados.

Figura 13: Arquitetura conceitual do stacking. Modelos base possivelmente diferentes produzem previsões para os mesmos exemplos. Essas previsões são usadas como entrada para um meta-modelo, que aprende a combiná-las para produzir a previsão final.

Pergunta 13

Um conjunto de dados tem 1.000 exemplos e 20 atributos. Um único Decision Tree treinado até a folha (sem poda) obtém 100% de acurácia no treino e 74% na validação. Um Random Forest com 100 árvores obtém 88% de acurácia na validação. **(a)** Por que o Random Forest generaliza melhor? **(b)** Qual mecanismo do Random Forest é responsável por diversificar as árvores e reduzir sua correlação? **(c)** Se você quisesse melhorar ainda mais o desempenho, como compararia Random Forest com Gradient Boosting para este problema?

Pergunta 14

As notas de AR compararam Q-learning tabular (exato, mas não escala) com Q-learning aproximado (generaliza, mas sem garantia de Q^*). Trace um paralelo preciso com a comparação entre árvores de decisão (exatas sobre o treino, sem regularização) e Random Forests (generalizadas, mas sem garantia de ótimo global). O que esses dois casos têm em comum conceitualmente? O que os distingue?

Tabela 2: Comparação entre métodos ensemble quanto ao componente do erro que atacam e à sua estratégia de combinação.

Método	**Reduz principalmente**	**Estratégia**
Bagging	Variância	Modelos independentes treinados em amostras bootstrap; combinação por votação ou média
Random Forest	Variância	Bagging com árvores + aleatoriedade na seleção de atributos
AdaBoost	Viés	Modelos sequenciais; exemplos errados recebem maior peso
Gradient Boosting	Viés; variância controlada por regularização	Modelos sequenciais treinados para corrigir resíduos do ensemble acumulado
Stacking	Depende dos modelos base e do meta-modelo	Modelos base produzem previsões; um meta-modelo aprende como combiná-las

E Aprofundamento em Redes Neurais

Este apêndice reúne propriedades de representação, funções de ativação e detalhes de treinamento que ampliam a apresentação do núcleo. Os papéis de camadas, não linearidade e retropropagação necessários à próxima nota já foram desenvolvidos na Seção 5.

E.1 Capacidade de representação

Por que redes neurais são universais. O *teorema de aproximação universal* ((Hornik, 1991; Cybenko, 1989)) afirma que um MLP com uma única camada oculta suficientemente larga pode aproximar, sob as hipóteses do teorema sobre a ativação e em um domínio compacto, qualquer função contínua com precisão arbitrária. Isso significa que a classe de hipóteses dos MLPs é extraordinariamente expressiva. A questão prática, no entanto, não é *se* um MLP pode representar a função, mas se ele consegue aprendê-la a partir de dados finitos e com gradiente descendente.

E.2 Funções de ativação

A escolha da função de ativação influencia fortemente a capacidade e a eficiência do treinamento. As mais usadas são:

- Sigmoide: $\sigma(z) = \frac{1}{1+e^{-z}} \in (0, 1)$. Historicamente importante, mas sofre de *vanishing gradient* em redes profundas.
- Tanh: $\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \in (-1, 1)$. Similar à sigmoide, mas centrada em zero; com vantagens e limitações que dependem da arquitetura.
- ReLU (*Rectified Linear Unit*): $\mathrm{ReLU}(z) = \max(0, z)$. Simples, eficiente, e muito usada em camadas ocultas; outras arquiteturas empregam outras ativações. Sua derivada é 1 para $z > 0$ e 0 para $z < 0$; unidades persistentemente inativas podem deixar de aprender. Isso não elimina todos os problemas de propagação de gradientes.
- Softmax (camada de saída para classificação multiclasse): $\mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$. Normaliza as saídas para uma distribuição de probabilidade sobre K classes.

O problema do gradiente que desaparece. A sigmoide comprime qualquer entrada para $(0, 1)$, e sua derivada máxima é 0,25. Em uma rede profunda, o gradiente é multiplicado por essa derivada em cada camada durante a retropropagação. O produto de dez derivadas sigmoidais é limitado por $0{,}25^{10} \approx 10^{-6}$. O gradiente completo também depende das matrizes de pesos e dos caminhos na rede; esse produto ilustra uma fonte de atenuação, não um limite geral para o gradiente completo. Isso faz com que os pesos das primeiras camadas aprendam muito lentamente ou nada. Esse problema, chamado de *vanishing gradient*, foi a principal barreira ao treinamento de redes profundas antes da adoção da ReLU.

E.3 Retropropagação

Como treinar os parâmetros de uma rede neural? O mesmo princípio do gradiente descendente se aplica, mas agora é necessário calcular o gradiente do custo em relação a

todos os pesos da rede. O algoritmo que faz isso de forma eficiente é a *retropropagação* (*backpropagation*).

A retropropagação não é um algoritmo de otimização; é um algoritmo de cálculo de gradientes. Ele aplica a *regra da cadeia do cálculo* de forma sistemática, propagando o erro da camada de saída de volta para a camada de entrada:

$$\frac{\partial J}{\partial w_{jk}^{(l)}} = \frac{\partial J}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial w_{jk}^{(l)}}$$

onde $a_j^{(l)}$ é a saída do neurônio j na camada l, incluindo a ativação e $w_{jk}^{(l)}$ é o peso que conecta o neurônio k da camada $l-1$ ao neurônio j da camada l.

A Figura 14 resume as duas fases computacionais usadas no treinamento de uma rede neural simples. No passo *forward*, os valores fluem da entrada x para a representação intermediária h, depois para a predição $\hat{y}$ e, por fim, para a função de perda J. Essa etapa calcula o erro produzido pelo modelo para os parâmetros atuais. No passo *backward*, o fluxo ocorre no sentido inverso: a partir da perda, calculam-se derivadas parciais sucessivas em relação às saídas intermediárias, como $\frac{\partial J}{\partial \hat{y}}$, $\frac{\partial J}{\partial h}$ e $\frac{\partial J}{\partial x}$. Essas derivadas são propagadas pela regra da cadeia e permitem calcular quanto cada peso e viés contribuiu para o erro. Assim, a retropropagação não é um novo modelo, mas um procedimento eficiente para obter os gradientes necessários à atualização dos parâmetros.

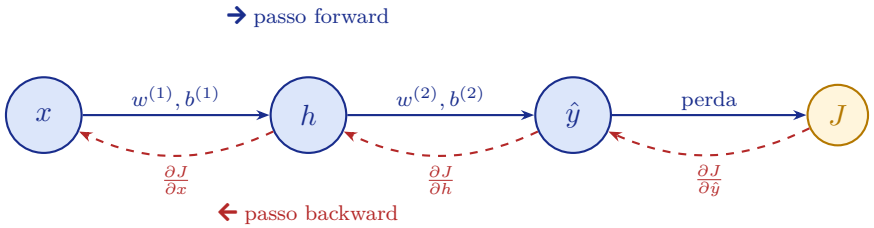


Figura 14: Intuição da retropropagação. No passo *forward*, os dados são propagados da esquerda para a direita para computar a perda J. No passo *backward*, os gradientes são propagados da direita para a esquerda via regra da cadeia, calculando $\frac{\partial J}{\partial w}$ para cada peso.

A retropropagação calcula os gradientes por diferenciação das operações da rede, à precisão numérica utilizada. Seu custo é da mesma ordem do passo forward; em um MLP denso, por exemplo e para uma arquitetura fixa, é da ordem do número de pesos W. Uma vez calculados, os gradientes são usados no gradiente descendente estocástico (SGD) para atualizar os pesos:

$$w_{jk}^{(l)} \leftarrow w_{jk}^{(l)} - \alpha \cdot \frac{\partial J}{\partial w_{jk}^{(l)}}$$

Retropropagação não é magia. Um equívoco comum é imaginar que a retropropagação "ajusta os pesos inteligentemente". Na realidade, ela faz uma coisa simples: aplicar a regra da cadeia sistematicamente para calcular derivadas parciais. O que há de elegante é a eficiência: em vez de estimar gradientes por diferenças finitas (custoso), a retropropagação os calcula pela regra da cadeia, à precisão numérica utilizada e com custo comparável ao forward.

A atualização dos pesos em si é o mesmo gradiente descendente da regressão linear. A única novidade é o cálculo eficiente dos gradientes numa função composta por muitas camadas.

E.4 Aprendizado profundo: uma visão geral

O *aprendizado profundo* (*deep learning*) é o nome dado ao uso de redes neurais com muitas camadas ocultas. A profundidade permite que as camadas intermediárias aprendam representações hierárquicas dos dados: na visão computacional, por exemplo, as primeiras camadas detectam bordas, as intermediárias detectam formas, e as últimas detectam objetos.

O sucesso do aprendizado profundo a partir de 2012 (com o AlexNet em ImageNet) foi impulsionado por três fatores:

- Dados: disponibilidade de conjuntos de dados maciços (ImageNet, Common Crawl, etc.).
- Computação: GPUs que permitem treinamento de redes grandes em tempo razoável.
- Algoritmos: melhores funções de ativação (ReLU), regularização (Dropout), normalização de lotes (*batch normalization*), e arquiteturas especializadas.

As arquiteturas mais influentes incluem:

- Redes convolucionais (CNNs): especializadas em dados com estrutura espacial (imagens).
- Redes recorrentes (RNNs / LSTMs): especializadas em sequências (texto, áudio, séries temporais).
- Transformers: arquitetura baseada em atenção, dominante em processamento de linguagem natural; base dos modelos GPT, BERT e Claude.

Aprendizado profundo e engenharia de características. Uma das contribuições mais importantes do aprendizado profundo é automatizar a engenharia de características: em vez de extrair manualmente $f_i(x)$, a rede aprende automaticamente quais características são relevantes como parte do treinamento. Isso explica por que modelos profundos, com dados suficientes, frequentemente superam modelos "rasos" com características manuais cuidadosamente projetadas.

Redes profundas não são solução universal. Redes neurais profundas têm alta capacidade, mas essa capacidade só é útil quando há dados, computação e regularização suficientes. Em bases pequenas ou tabulares, modelos mais simples (como regressão logística, Random Forests ou Gradient Boosting) frequentemente apresentam desempenho equivalente ou superior, com muito menos custo computacional e maior interpretabilidade. A escolha do modelo deve ser guiada pelo problema, não pela complexidade.

Pergunta 15

Explique em uma frase o que a retropropagação faz. Descreva o papel da função de ativação numa rede neural: o que aconteceria se todas as ativações fossem lineares? Qual arquitetura de rede neural seria mais adequada para classificar imagens de raio-X médico?

Pergunta 16

Considere uma rede neural com uma camada oculta de 10 neurônios ReLU treinada para classificação binária (spam/não-spam). O modelo obtém 99% de acurácia no treino mas apenas 65% na validação. **(a)** Qual problema está ocorrendo? **(b)** Cite três técnicas específicas de treinamento de redes neurais que poderiam melhorar a generalização. **(c)** Como o conceito de regularização (Seção 3) se aplica a redes neurais?

F Aprendizado Não Supervisionado

F.1 Quando não temos rótulos

Nos problemas de aprendizado supervisionado, cada exemplo de treinamento vem acompanhado de um rótulo y. Na prática, rotular dados é caro, demorado e muitas vezes impraticável em larga escala. O *aprendizado não supervisionado* aborda o problema de extrair estrutura de dados *sem rótulos*.

As tarefas mais comuns são:

- Agrupamento (*clustering*): encontrar grupos naturais nos dados.
- Redução de dimensionalidade: representar os dados em um espaço de menor dimensão, preservando estrutura relevante.
- Estimação de densidade: modelar a distribuição $p(\mathbf{x})$.

F.2 k-means

O algoritmo k-means é o método de agrupamento mais simples e amplamente usado. Dado um conjunto de pontos $\{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n)}\}$ e um número de grupos k, o k-means encontra k *centróides* $\{\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k\}$ que minimizam a soma das distâncias quadráticas de cada ponto ao seu centróide mais próximo:

$$J(\{C_i\}, \{\boldsymbol{\mu}_i\}) = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$

O algoritmo alterna entre dois passos até convergência:

Algoritmo: k-means.

1. Inicialize k centróides $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_k$ (por exemplo, com pontos distintos do conjunto).
2. **Atribuição (passo E):** para cada ponto, escolha
$$c^{(i)} \leftarrow \arg\min_j \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2.$$
3. **Atualização (passo M):** para cada grupo não vazio, calcule
$$\boldsymbol{\mu}_j \leftarrow \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}.$$
4. Repita os dois passos até estabilizar as atribuições; devolva os centróides e os grupos. Se um grupo ficar vazio, é preciso adotar uma regra, como reinicializar seu centróide. Use uma regra consistente para desempates.

A Figura 15 ilustra o funcionamento do algoritmo k-means para $k = 3$. No painel inicial, os pontos ainda não possuem grupos definidos, e os centróides começam em posições escolhidas aleatoriamente, indicadas pelas estrelas. A cada iteração, o algoritmo executa dois passos: primeiro, atribui cada ponto ao centróide mais próximo; depois, reposiciona cada centróide na média dos pontos atribuídos ao seu grupo. Esse processo se repete até que as atribuições deixem de mudar de forma relevante. No painel final, os pontos aparecem separados em três agrupamentos estáveis, cada um representado por uma cor e por seu centróide final. A figura também sugere uma limitação importante: como o resultado depende dos centróides iniciais, diferentes inicializações podem levar a agrupamentos diferentes.

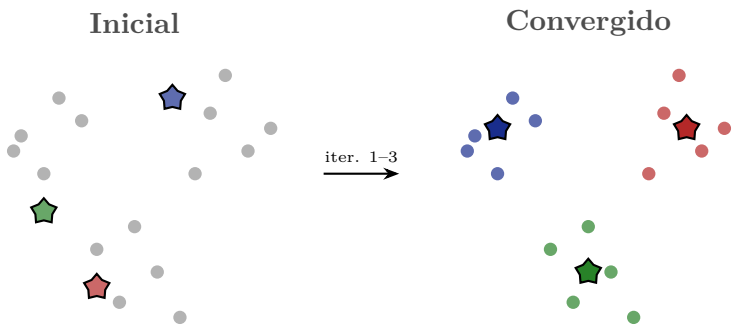


Figura 15: O algoritmo k-means com $k = 3$. Partindo de centróides iniciais aleatórios (estrelas), o algoritmo alterna entre atribuição de pontos ao centróide mais próximo e recálculo dos centróides, convergindo para um agrupamento estável.

F.2.1 Exemplo resolvido

Aplique k-means com $k = 2$ ao conjunto $\{(1,1),(1,2),(5,4),(5,5)\}$. Centróides iniciais: $\boldsymbol{\mu}_1 = (1,1)$ e $\boldsymbol{\mu}_2 = (5,4)$.

Iteração 1 – Passo E:

- $(1,1)$: dist. a $\boldsymbol{\mu}_1 = 0$; dist. a $\boldsymbol{\mu}_2 = \sqrt{32} \approx 5{,}7$. Atribui ao grupo 1.
- $(1,2)$: dist. a $\boldsymbol{\mu}_1 = 1$; dist. a $\boldsymbol{\mu}_2 = \sqrt{25} = 5$. Atribui ao grupo 1.
- $(5,4)$: dist. a $\boldsymbol{\mu}_1 = \sqrt{25} = 5$; dist. a $\boldsymbol{\mu}_2 = 0$. Atribui ao grupo 2.
- $(5,5)$: dist. a $\boldsymbol{\mu}_1 = \sqrt{32} \approx 5{,}7$; dist. a $\boldsymbol{\mu}_2 = 1$. Atribui ao grupo 2.

Iteração 1 – Passo M:

$$\boldsymbol{\mu}_1 = \frac{(1,1)+(1,2)}{2} = (1,\ 1{,}5) \qquad \boldsymbol{\mu}_2 = \frac{(5,4)+(5,5)}{2} = (5,\ 4{,}5)$$

Iteração 2 – Passo E: as atribuições não mudam. Algoritmo converge.

Interpretação: os dois grupos naturais $\{(1,1),(1,2)\}$ e $\{(5,4),(5,5)\}$ foram corretamente identificados em duas iterações.

Limitações do k-means. O k-means tem limitações importantes: **(a)** requer que k seja fornecido a priori; **(b)** converge para um mínimo local que depende da inicialização (o algoritmo *k-means++* melhora a inicialização); **(c)** assume grupos de forma esférica e variância similar; falha com grupos alongados, em forma de lua crescente, ou com densidade muito diferente.

F.3 Análise de Componentes Principais (PCA)

A *análise de componentes principais* (PCA, *Principal Component Analysis*) é o método de redução de dimensionalidade mais clássico. Dado um conjunto de dados em $\mathbb{R}^d$, o PCA encontra uma projeção linear para um espaço de dimensão $k < d$ que *maximiza a variância preservada* (equivalentemente, minimiza o erro de reconstrução).

A Figura 16 mostra a intuição geométrica da Análise de Componentes Principais em duas dimensões. A nuvem de pontos possui uma direção dominante de espalhamento: os dados variam mais ao longo do eixo indicado por PC_1. Essa é a primeira componente

principal. A segunda componente, PC_2, é perpendicular à primeira e captura a maior parte da variação que ainda resta depois de PC_1. Quando projetamos os dados apenas sobre PC_1, substituímos cada ponto bidimensional por uma única coordenada ao longo dessa direção. Assim, reduzimos a dimensionalidade de 2 para 1 preservando o máximo possível da variância dos dados. A figura também ajuda a lembrar que PCA não usa rótulos de classe: ele procura direções de maior variância, não necessariamente direções de melhor separação entre grupos.

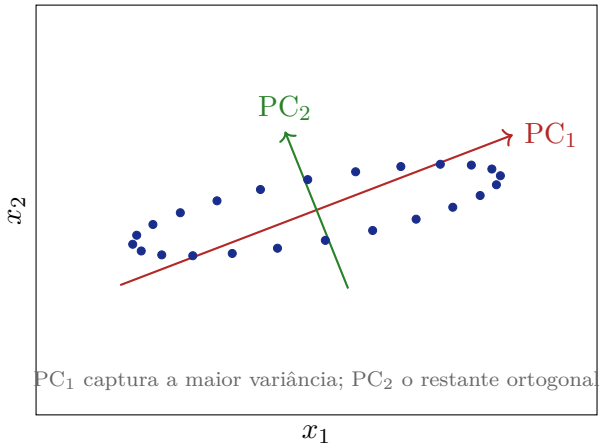


Figura 16: PCA em 2D. Os dados formam uma nuvem elíptica. A primeira componente principal (PC_1) aponta na direção de maior variância; a segunda (PC_2) é perpendicular a ela. Projetar os dados sobre apenas PC_1 reduz a dimensão de 2 para 1, com mínima perda de informação.

Intuitivamente, o PCA encontra as direções do espaço de características ao longo das quais os dados variam mais. Projetar os dados sobre as k primeiras componentes principais retém a maior parte da variância com apenas k dimensões.

O PCA tem múltiplas aplicações práticas:

- Visualização: projetar dados de alta dimensão para 2D ou 3D.
- Pré-processamento: reduzir dimensionalidade antes de aplicar outro algoritmo de AM, acelerando o treinamento.
- Remoção de ruído: as componentes de menor variância frequentemente capturam ruído; descartá-las pode melhorar o sinal.
- Compressão: representar imagens ou documentos de forma compacta.

PCA e redes neurais. O PCA computa uma projeção linear ótima. Autocodificadores (*autoencoders*) são redes neurais treinadas para comprimir e reconstruir dados. Sob condições adequadas (dados centrados, gargalo linear, perda quadrática e solução ótima), um autocodificador linear aprende o mesmo subespaço principal do PCA; suas coordenadas não precisam coincidir com as componentes principais. Com ativações não lineares, o autocodificador aprende projeções não lineares, uma generalização poderosa do PCA.

Pergunta 17

Qual a diferença fundamental entre aprendizado supervisionado e não supervisionado? O que o k-means minimiza? Por que é necessário especificar k a priori, e como você poderia escolher um bom valor de k sem conhecimento prévio?

Pergunta 18

Uma empresa de varejo quer segmentar seus clientes em grupos para personalizar campanhas de marketing. Os dados disponíveis incluem: valor médio de compra, frequência de visitas mensais, categorias de produtos comprados (como um vetor de 50 dimensões representando proporção de compras em cada categoria) e informações demográficas. **(a)** Argumente por que esta é uma tarefa de aprendizado não supervisionado, e não de aprendizado supervisionado. **(b)** Quais são as limitações do k-means para este problema? **(c)** Por que seria útil aplicar PCA antes do k-means neste caso?

G Panorama Comparativo das Famílias de Modelos

O AM é o estudo de algoritmos que aprendem a partir de dados. O framework supervisionado formaliza o aprendizado como minimização de um risco empírico num espaço de hipóteses, sujeito à restrição implícita de generalização. O dilema viés-variância capta a tensão fundamental entre modelos simples (que generalizam mas erram sistematicamente) e modelos complexos (que ajustam bem o treino mas podem não generalizar). A regularização, a validação cruzada e a divisão adequada dos dados são os instrumentos práticos para navegar esse dilema.

Modelos lineares são simples, eficientes e interpretáveis, mas limitados. Árvores de decisão são interpretáveis mas instáveis; métodos ensemble as tornam robustas. Redes neurais têm grande capacidade expressiva sob condições apropriadas e aprendem características automaticamente, ao custo de requisitos maiores de dados, computação e conhecimento de treinamento. Cada família de modelos tem seu lugar, e a escolha deve ser guiada pela natureza do problema, pelo volume e qualidade dos dados, e pelos requisitos de interpretabilidade e desempenho.

Não existe almoço grátis (*no free lunch*). O teorema "no free lunch" estabelece que, em geral, não existe um algoritmo de AM universalmente melhor que todos os outros. Todo algoritmo faz suposições implícitas sobre a estrutura do problema (o que é chamado de *viés indutivo* (*inductive bias*) do algoritmo). Um algoritmo eficiente para um tipo de problema pode ser ineficiente para outro.

Na prática, isso significa que a escolha do modelo (i.e., de $\mathcal{H}$) deve ser informada pelo conhecimento do domínio e validada empiricamente. Não existe o modelo "sempre melhor"; existe o modelo mais adequado para o problema e os dados disponíveis.

A lista a seguir sintetiza os modelos do núcleo e dos apêndices, destacando paradigma, tipo de função aprendida, perfil viés-variância e observações práticas.

- **Regressão linear**
  - **Paradigma:** supervisionado.
  - **O que aprende:** uma função linear de predição.
  - **Viés/variância:** frequentemente alto viés em relações complexas e menor variância; depende dos dados e da regularização.
  - **Escalabilidade/notas:** admite solução analítica quando d é pequeno; pode ser treinada com SGD quando d é grande.
- **Regressão logística**
  - **Paradigma:** supervisionado.
  - **O que aprende:** uma fronteira linear de decisão para classificação.
  - **Viés/variância:** frequentemente alto viés em relações complexas e menor variância; depende dos dados e da regularização.
  - **Escalabilidade/notas:** eficiente, interpretável e base para o classificador softmax.
- **Árvore de decisão**
  - **Paradigma:** supervisionado.

  - **O que aprende:** um particionamento hierárquico do espaço de entrada.
  - **Viés/variância:** tipicamente baixo viés e alta variância em árvores profundas.
  - **Escalabilidade/notas:** interpretável, mas instável; tende a overfitting sem poda ou outras formas de regularização.

- **Random Forest**
  - **Paradigma:** supervisionado.
  - **O que aprende:** um ensemble de árvores de decisão.
  - **Viés/variância:** reduz a variância em relação a uma única árvore.
  - **Escalabilidade/notas:** robusto, pouco sensível a hiperparâmetros e escalável por paralelismo entre árvores.

- **Gradient Boosting**
  - **Paradigma:** supervisionado.
  - **O que aprende:** um ensemble sequencial de modelos fracos.
  - **Viés/variância:** pode reduzir viés e variância.
  - **Escalabilidade/notas:** muito poderoso, mas mais sensível a hiperparâmetros.

- **MLP / Deep Learning**
  - **Paradigma:** supervisionado.
  - **O que aprende:** representações hierárquicas dos dados.
  - **Viés/variância:** depende da arquitetura, do tamanho do modelo, da regularização e dos dados.
  - **Escalabilidade/notas:** alta capacidade expressiva; requer muitos dados e treinamento computacionalmente custoso.

- **k-means**
  - **Paradigma:** não supervisionado.
  - **O que aprende:** centróides que representam grupos nos dados.
  - **Viés/variância:** não se aplica diretamente no mesmo sentido usado para modelos supervisionados.
  - **Escalabilidade/notas:** sensível à inicialização e à escolha de k; assume implicitamente clusters aproximadamente esféricos.

- **PCA**
  - **Paradigma:** não supervisionado.
  - **O que aprende:** uma projeção linear nas direções de maior variância.
  - **Viés/variância:** não se aplica diretamente no mesmo sentido usado para modelos supervisionados.
  - **Escalabilidade/notas:** fornece uma projeção linear ótima em termos de variância preservada; é eficiente e serve de base para muitas técnicas.

H Atividades Opcionais de Aprofundamento

Estas atividades preservam o roteiro ampliado da versão anterior. Não integram as atividades obrigatórias dos três encontros. Consulte os apêndices correspondentes antes de resolvê-las.

As atividades abaixo podem ser resolvidas em grupo, com foco em explicar o raciocínio passo a passo, e não apenas em obter a resposta final.

H.1 Aquecimento

Antes das atividades principais, responda rapidamente:

(a) Qual a diferença entre um parâmetro (como os pesos $\mathbf{w}$) e um hiperparâmetro (como λ na regularização)? O que é ajustado durante o treinamento e o que é ajustado durante a validação?

(b) Um modelo obteve erro de treino = 0,02 e erro de validação = 0,85. Qual problema você diagnosticaria? E se o erro de treino fosse 0,80 e o de validação 0,82?

(c) Cite a diferença entre MSE e entropia cruzada. Para qual tarefa cada uma é mais adequada?

(d) Explique a diferença entre bagging e boosting em uma frase cada.

(e) Por que inicializar todos os pesos de uma rede neural com zero é problemático? (*Dica*: pense no que acontece com o gradiente se todos os neurônios de uma camada tiverem os mesmos pesos.)

(f) Em k-means, o que o algoritmo está minimizando? O que garante convergência?

Pergunta 19

Explique para um colega a diferença entre viés e variância usando apenas:

- a analogia de um arqueiro num campo de tiro;
- o comportamento do modelo com dados de treino vs. dados de validação;
- um exemplo concreto com regressão linear de grau 1 vs. grau 15;
- por que um modelo com alto viés e um com alta variância cometem erros muito diferentes.

Evite fórmulas. Para aprofundar a explicação, consulte a Seção 3 e o Apêndice B.

H.2 Regressão linear e gradiente descendente

Considere um problema de regressão com os dados a seguir:

Exemplo	Área (x_1, mš)	Quartos (x_2)	Preço (y, R$ mil)
1	50	1	200
2	80	2	300
3	120	3	450
4	60	2	250

(a) Escreva a hipótese do modelo: $h(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2$. Calcule o MSE para os pesos iniciais $w_0 = 0$, $w_1 = 3$, $w_2 = 10$.

(b) Calcule o gradiente $\frac{\partial J}{\partial w_1}$ e $\frac{\partial J}{\partial w_2}$ com os pesos acima. (Use $\frac{\partial J}{\partial w_j} = \frac{2}{n}\sum_i (\hat{y}_i - y_i) x_{ij}$.)

(c) Execute um passo de gradiente descendente com $\alpha = 0{,}0001$ e reporte os pesos atualizados.

(d) Após o passo, o MSE aumentou ou diminuiu? Por que isso era esperado?

H.3 Entropia e construção de árvores

Considere um conjunto de 12 exemplos de e-mails: 8 spam e 4 legítimos. Dois atributos candidatos para a divisão raiz:

- **Atributo A** ("oferta" presente): $S_{\text{sim}} = \{6\text{spam}, 1\text{leg}\}$, $S_{\text{não}} = \{2\text{spam}, 3\text{leg}\}$.
- **Atributo B** (enviado de noite): $S_{\text{sim}} = \{4\text{spam}, 2\text{leg}\}$, $S_{\text{não}} = \{4\text{spam}, 2\text{leg}\}$.

(a) Calcule a entropia do conjunto raiz $H(S)$.

(b) Calcule o ganho de informação $\text{IG}(S, A)$ e $\text{IG}(S, B)$.

(c) Qual atributo o algoritmo de árvore escolheria? Justifique intuitivamente.

(d) O Atributo B tem IG igual a zero. O que isso diz sobre o poder discriminativo desse atributo? (Calcule para confirmar.)

H.4 Diagnóstico de generalização

Para cada cenário abaixo, diagnostique o problema (underfitting, overfitting, ou adequado), identifique o provável desequilíbrio viés-variância e proponha uma ação corretiva específica.

(a) Um modelo de regressão linear simples ($d = 1$, sem termos polinomiais) para prever o consumo de energia em função da temperatura, com erro de treino = 320 e erro de validação = 335.

(b) Uma Random Forest com 500 árvores, profundidade máxima ilimitada, treinada em 200 exemplos, com erro de treino = 0 e erro de validação = 0,45 (acurácia de 55%).

(c) Uma regressão logística com regularização ℓ_2, $\lambda = 1000$, com acurácia de treino = 53% e de validação = 52%.

(d) Um MLP com 5 camadas ocultas de 1.000 neurônios, treinado por 500 épocas, com perda de treino = 0,001 e perda de validação = 2,8.

H.5 k-means: execução manual

Aplique o algoritmo k-means com $k = 3$ ao conjunto de pontos em 2D (coordenadas inteiras):

Ponto	Coordenadas
A	(1, 1)
B	(1, 2)
C	(2, 1)
D	(8, 8)
E	(9, 8)
F	(8, 9)
G	(4, 5)
H	(5, 4)
I	(5, 5)

Centróides iniciais: $\boldsymbol{\mu}_1 = (1, 1)$, $\boldsymbol{\mu}_2 = (4, 5)$, $\boldsymbol{\mu}_3 = (8, 8)$.

(a) Execute a iteração 1 completa: passo E (atribuição) e passo M (atualização dos centróides). Use distância euclidiana.

(b) Execute a iteração 2 e verifique se o algoritmo convergiu.

(c) O custo J diminuiu da iteração 1 para a iteração 2? Por que o custo nunca aumenta entre iterações no k-means?

H.6 Perguntas para discussão em sala

Pergunta 20

Considere o seguinte cenário: você tem 100.000 e-mails rotulados (spam/legítimo) e precisa escolher entre três modelos: regressão logística, Random Forest e MLP com 3 camadas. Descreva o processo completo de desenvolvimento e avaliação, desde a divisão dos dados até a implantação do modelo. Que critérios usaria para a escolha final entre os três modelos?

Pergunta 21

O dilema viés-variância implica que não existe um modelo universalmente melhor. No entanto, nas competições Kaggle de AM, Gradient Boosting (XGBoost/-LightGBM) e redes neurais dominam consistentemente. Como reconciliar o teorema "no free lunch" com o sucesso prático desses métodos específicos em tantos problemas? (*Dica*: considere que competições Kaggle não amostram uniformemente todos os problemas possíveis de AM; elas concentram certos tipos de dados, métricas e tarefas.)

Pergunta 22

Um sistema de concessão de crédito usa um modelo de AM para decidir se um empréstimo deve ser aprovado. O modelo foi treinado com dados históricos de uma instituição financeira. **(a)** Quais riscos surgem se a hipótese de aprendizagem não for satisfeita nesse contexto? **(b)** Por que a acurácia pode ser uma métrica inadequada? **(c)** Que considerações éticas surgem quando os dados históricos refletem vieses sociais existentes?

Pergunta 23

Na aproximação linear de Q-learning, o alvo TD é tratado como constante na atualização, que constitui um semigradiente. À luz do que foi estudado sobre AM, identifique: **(a)** o "conjunto de treinamento" equivalente no Q-learning; **(b)** o "rótulo" (y) equivalente; **(c)** de que forma o problema de generalização aparece no Q-learning com aproximação, e por que ele assume uma forma diferente do overfitting supervisionado clássico; **(d)** que tipo de regularização seria natural aplicar ao Q-learning com aproximação.

Referências

Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.

Chen, T. e Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2, 303–314.

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257.